JavaScript

bstorm TRAINING

Qui sommes-nous ?

Suivez-nous sur

Brainstorm Consulting srl
Rue du Bosquet, 7
1348 Louvain-la-Neuve
info@bstorm.be
010 24 71 10

bstorm TRAINING	bstorm est un acteur reconnu de la formation professionnelle en Belgique, avec plus de 4000 journées de formation délivrées chaque année. Nous accompagnons les entreprises dans la montée en compétences de leurs équipes à travers des parcours sur mesure en développement logiciel (.NET, Java, JavaScript, Python), en front-end (React), en DevOps, en **intelligence artificielle**, en analyse de données (SQL, Power BI), en business analysis et en gestion de projet.
bstorm DATA	bstorm met l'analyse des données au service de la performance opérationnelle. Nous aidons les organisations à **exploiter pleinement leur capital data** via des dashboards interactifs, **des modèles prédictifs, du machine learning et des outils d'aide à la décision**. Notre approche privilégie la simplicité, la pertinence et **l'autonomie** dans l'usage quotidien des données, pour des décisions plus rapides et mieux informées.
bstorm DIGITAL TRANSFORMATION	bstorm accompagne les entreprises dans l'ensemble de leur transformation digitale : diagnostic, automatisation des processus, dématérialisation, modernisation des outils métier, gestion documentaire, collaboration, cybersécurité, intégration de l'IA et pilotage par les données. Nous combinons conseil stratégique, accompagnement au changement et **développement de solutions concrètes pour renforcer l'efficacité opérationnelle et la résilience des organisations.**
bstorm ARTIFICIAL INTELLIGENCE	bstorm Artificial Intelligence accompagne les entreprises dans l'identification et la mise en œuvre de cas d'usage concrets de l'IA. De la **sélection des outils aux intégrations techniques**, nous développons des solutions sur mesure pour **automatiser, prédire, assister ou optimiser vos processus.** Notre mission : **rendre l'IA accessible**, utile et directement applicable à vos métiers.

Table des matières

1. Introduction
   a. Le JavaScript, c'est quoi ?
   b. Premiers pas en JavaScript
2. Les variables
   a. Déclaration de variable
   b. Le typage
   c. Les valeurs prédéfinies
3. Les types de données
   a. Number
   b. BigInt
   c. String
   d. Boolean
   e. Object
4. Interaction avec le DOM - Les Bases
   a. Introduction au DOM
   b. Interaction basique avec une page
5. Les opérateurs de condition
   a. Les opérateurs d'égalité
   b. Les opérateurs de comparaison
   c. Les opérateurs logique
6. Traitement conditionnelle
   a. La structure « if...else »
   b. La structure « switch »
   c. L'opérateur « ternaire »
   d. Les opérateurs logiques
   e. L'opérateur « nullish coalescing »

Table des matières

7. Les Dates
   a. L'objet de type « Date »
   b. Méthodes d'un objet « Date »
8. Interaction avec le DOM - Modifier la page
   a. Créer un nouvel élément
   b. Les méthodes d'interaction
9. Traitement itératif
   a. La structure « while »
   b. La structure « do...while »
   c. La structure « for »
   d. Instructions « continue » et « break »
10. Les collections indexées
   a. Déclaration et utilisation
   b. Les méthodes des collections
11. Traitement itératif - Les collections
   a. Les structures itératives
   b. La méthode « Array.foreach »
   c. La structure « for...of »
   d. La structure « for...in »
12. Les fonctions
   a. Déclaration et utilisation
   b. Les paramètres
   c. Les fonctions fléchées
   d. Les « Callback »

Table des matières

13. Les timers
    a. Timeout
    b. Interval
14. Interaction avec le DOM - Les événements
    a. Abonnement aux events
    b. Variable de l'événement
    c. Types d'events des éléments du DOM
    d. Types d'events de « Window »
15. Les exceptions
    a. L'instruction « try ... catch ... finally »
    b. Déclencher une exception
16. Interaction avec le DOM - Les formulaires
    a. Objectif des formulaires
    b. Manipulation les formulaires
17. Les promesses
    a. Déclaration et utilisation
    b. Les méthodes des promesses
    c. Les fonctions asynchrones
18. Annexes
    a. Les collections à clefs : Map & Set
    b. Évaluation en court-circuit
    c. Les closures
    d. Le destructuring
    e. Web Storage
    f. L'API de validation des formulaires
19. Références

Introduction

Le JavaScript, c'est quoi ?

Introduction

Le langage JavaScript

Créé en 1995 par Brendan Eich et intégré au navigateur Netscape, celui-ci est utilisé pour permettre de réaliser la validation des formulaires par le navigateur.

Ensuite, il a rapidement évolué pour devenir un langage puissant et polyvalent.

Quelques particularités du langage :

- Le code du JavaScript est interprété ou compilé à la volée.
- Il dispose d'un typage faible et dynamique.
- Il utilise le concept d'objet à prototype.

Utilisation du langage JavaScript

Bien que principalement connu en tant que langage de script Web, le langage Javascript n'est plus limité à une utilisation au sein d'un navigateur Web.

Aujourd'hui, le JavaScript permet de réaliser, par exemple :

- Site web dynamique (formulaire interactif, animation 3D, jeux vidéo ...)
- Application client Web, à l'aide des framework tels que Next JS, Angular,...
- Application server, grâce à des plateformes telles que Node JS, Deno, Bun...
- Application Mobile, en utilisant les framework React Native, Ionic,...

Le standard ECMAScript

En 1997, le JavaScript est standardisé par la norme ECMAScript (ECMA-262).

Cette norme permet de définir :

- La syntaxe du langage.
- Les types de variables.
- Les objets et fonctions natives.
- Le mécanisme de traitement des erreurs.
- Etc...

Depuis 2015, le rythme de publication de la norme ECMAScript est annuel.

Site Web utile

- La documentation du JavaScript - MDN Web Docs
  https://developer.mozilla.org/fr/docs/Web/JavaScript
- Spécification du langage ECMAScript
  https://www.ecma-international.org/publications-and-standards/standards/ecma-262/
- Contribuer à la norme ECMAScript
  https://tc39.es/
- Vérifier la compatibilité des navigateurs
  https://caniuse.com/?search=ECMAScript

Développer en JavaScript

Dans le cadre du cours, nous allons faire du JavaScript destiné à une page Web.
Il nous faudra donc les deux outils suivants : Un éditeur de texte et un navigateur.

Il existe plusieurs éditeurs de texte adaptés au JavaScript :

- Visual Studio Code
- WebStorm (gratuit pour une utilisation non commerciale)
- Vim
- Sublime Text
- ...

Premiers pas en JavaScript

Introduction

Emplacement du code

Pour intégrer du code JavaScript dans une page web, il y a plusieurs possibilités :

- Directement dans la page HTML

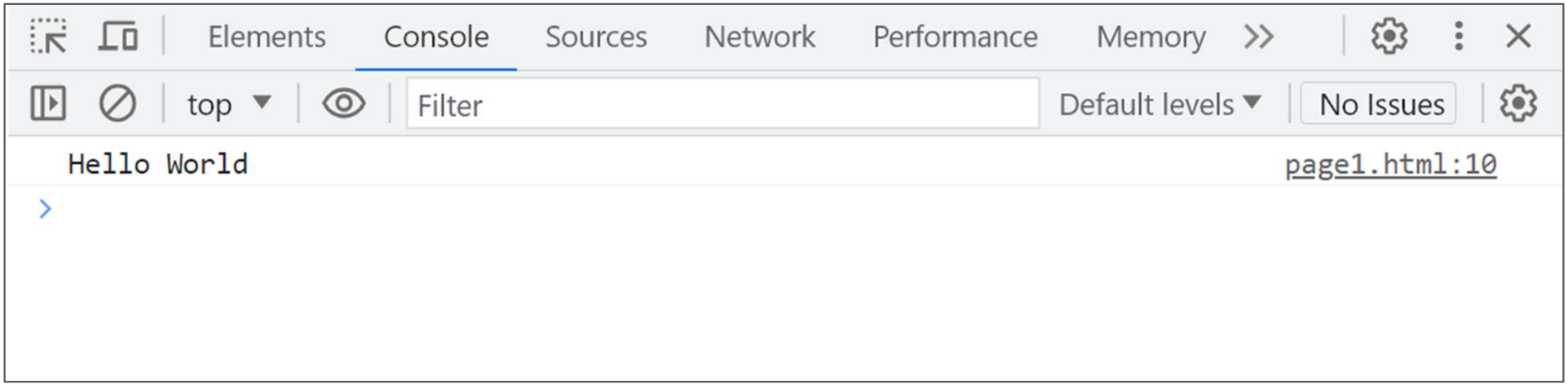
```
<script>
  const message = "Hello World";
  console.log(message);
</script>
```

- Depuis un fichier « js »

```
<script src="js/script.js" defer></script>
```

Déboguer son code

Les navigateurs web permettent d'avoir accès à un outil de développement.



La console permet de voir les messages envoyés par le code ou d'en exécuter.

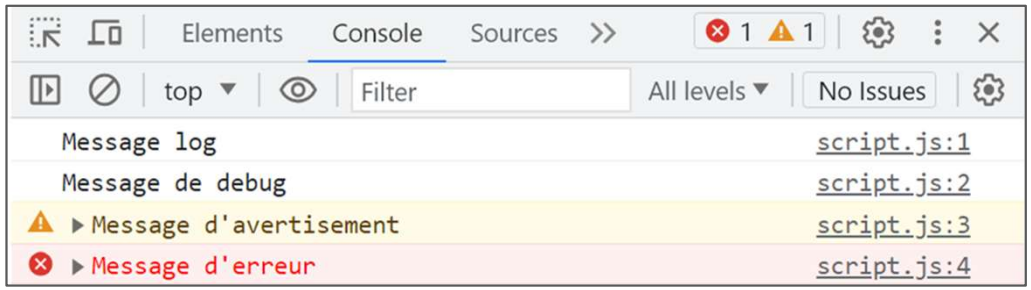
Les raccourcis habituels pour l'ouvrir sont « F12 » ou « Ctrl Maj i ».

Utilisation de la console

Afficher un message dans la console :

```
console.log('Message log');
console.debug('Message de debug');
console.warn('Message d\'avertisement');
console.error('Message d\'erreur');
```

Résultat dans la console :



Les différentes méthodes permettent d'afficher un message dans la console avec un niveau de criticalité.

La console permet d'ajouter un filtre sur le niveau de criticalité des messages.

Interaction avec le navigateur

Les navigateurs Web permettent d'afficher des popups depuis le JavaScript.

Cela permet de :

- Afficher un message
- Réaliser une saisie
- Obtenir une confirmation

Bien que ces méthodes d'interaction soient simple à utiliser, il ne faut pas en abuser.
Il sera toujours préférable d'interagir à l'aide de la page Web (via l'API du DOM), pour offrir une meilleur expérience à l'utilisateur.

Interaction avec le navigateur

Afficher un message via le navigateur :

```
alert('Hello World ♥');
```

Résultat dans le navigateur :

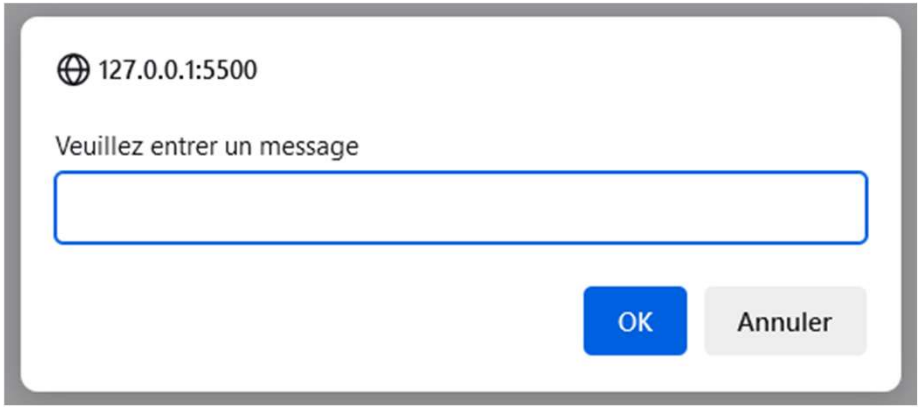


Interaction avec le navigateur

Permettre la saisie de l'utilisateur via le navigateur :

```
prompt('Veuillez entrer un message');
```

Résultat dans le navigateur :



Interaction avec le navigateur

Obtenir une confirmation de l'utilisateur via le navigateur :

```
confirm('C\'est bon pour vous ?');
```

Résultat dans le navigateur :



Le mode strict

Depuis l'ECMAScript 5, il est possible d'utiliser une variante restrictive du JavaScript.

Cette variante est appelée le « Strict Mode », elle apporte les changements suivants :

- Élimination d'erreurs silencieuses, en les transformant en erreurs explicites.
- Interdiction de mots-clefs susceptibles d'être utilisés dans les futures versions.
- Correction des erreurs pour permettre une meilleure optimisation par les moteurs JavaScript *(Un même code sera donc exécuté plus rapidement en mode strict)*.

Pour l'activer sur un fichier, ajouter « **'use strict';** » en première instruction.

Les commentaires

Ajouter des commentaires dans votre code JavaScript permettra de documenter votre code pour que celui-ci reste compréhensible.

- Commentaire sur une ligne

```
// Un commentaire
```

- Commentaire sur plusieurs lignes

```
/* Commentaire
sur plusieurs
lignes */
```

Bien évidemment, les commentaires ne sont pas interprétés par le moteur JavaScript.

Les variables

Déclaration de variable

Les variables

Déclaration de variable

Il existe trois manières de déclarer des variables en JavaScript :

- Variable globale modifiable « **var** »
- Variable locale modifiable « **let** »
- Variable locale immuable « **const** »

	Global	Portée limitée à la fonction	Portée limitée au bloc	Ré-assignable	Re-déclarable	Hoisting
var	✔	✔	✘	✔	✔	✔
let	✘	✔	✔	✔	✘	✘
const	✘	✔	✔	✘	✘	✘

Nommer une variable

Le nom d'une variable peut contenir les caractères suivants :

- des caractères alphanumériques
- des underscores
- des dollars

Le nom de la variable doit commencer par un dollar, un underscore ou une lettre.

La convention de nommage pour les variables est le « camelCase ».

Le typage

Les variables

Le typage

Le JavaScript est un langage à typage dynamique.

Le type d'une variable sera défini dynamiquement lors de l'exécution en fonction du type du contenu qu'elle stockera.

De plus, il est possible que le type d'une variable change durant l'exécution

Bonne pratique :
Ne pas utiliser une même variable pour stocker plusieurs types de données !

Les types de valeurs

En JavaScript, ce sont des « pseudo-objets » qui permettent de définir les différents types de donnée avec leurs propriétés et leurs méthodes.

- Number : Un nombre (entier ou réel)
- BigInt : Un grand nombre entier
- String : Une chaîne de caractères
- Boolean : Type dont les valeurs possibles sont « true » ou « false »
- Undefined : Variable non initialisée
- Null : Variable initialisée sans aucune valeur
- Object : Élément complexe possédant plusieurs valeurs

Aperçu de quelques types de données

- Un nombre entier

```
const numberInteger = 42;
```

- Un nombre réel

```
const numberReal = 3.14;
```

- Une chaîne de caractères

```
const firstname = 'Della';
```

- Un booléen

```
const isValid = true;
```

- Un tableau de nombre

```
const tabNumber = [13, 42, 99, 42];
```

- Un tableau de chaîne de caractères

```
const tabName = ['Riri', 'Fifi', 'Zaza'];
```

- Une valeur « null »

```
const example = null;
```

- Un objet

```
const person = {
  firstname : 'Balthazar',
  lastname : 'Picsou'
};
```

Obtenir le type d'une variable

L'opérateur « typeof » permet d'obtenir le type d'une variable.

Le type est renvoyé sous la forme d'une chaîne de caractères.

```
const response = 42;
const responseType = typeof response;

console.log(responseType); // → 'number'
```

Remarque : L'utilisation de l'opérateur « typeof » sur une variable de type « Null » renvoie la valeur « 'object' » pour des raisons historiques.

Obtenir le type d'une variable - Les objets

L'opérateur « typeof » ne peut pas indiquer le prototype d'un objet (tableau, date, ...). Celui-ci renverra toujours le résultat « 'object' ».

Pour connaître le prototype d'une variable de type « Object », il est possible :

- D'obtenir le nom du prototype via la propriété « constructor »
- De tester l'appartenance à un prototype via l'opérateur « instanceof »

```
const now = new Date();

console.log(typeof now);           // → 'object'
console.log(now.constructor.name); // → 'Date'
console.log(now instanceof Date);  // → true
```

Les valeurs prédéfinies

Les variables

Undefined

La valeur « undefined » est une propriété globale qui représente la primitive undefined.

Cette valeur primitive est affectée automatiquement :

- aux variables qui viennent d'être déclarées sans avoir été initialisée.
- aux paramètres de fonction qui ne possèdent pas de valeur par défaut.

Null

La valeur « null » est un littéral qui représente l'absence intentionnelle de valeur pour une variable objet. C'est une valeur primitive du langage JavaScript.

Contrairement à la valeur « undefined », ce n'est pas une propriété globale.

NaN

La valeur « NaN » (Not a Number) est une propriété globale qui permet de représenter une valeur numérique invalide.

Particularité de cette propriété « NaN » :

- Elle est de type « 'number' »

```
console.log(typeof NaN);     // → 'number'
```

- Les opérateurs d'égalité ne peuvent pas être utilisés sur celle-ci.
  Pour tester cette valeur, il est possible d'utiliser la méthode globale « isNaN(...) »

```
console.log(NaN === NaN);    // → false
console.log(isNaN(NaN));     // → true
```

Infinity

La valeur « Infinity » est une propriété globale qui permet de représenter l'infini.

Comportement de la valeur « Infinity » :

- Tout nombre multiplié par Infinity sera égal à Infinity
- Zéro multiplié par Infinity sera égal à NaN
- NaN multiplié par Infinity sera égal à NaN
- Tout nombre divisé par Infinity sera égal à zéro
- Infinity divisé par Infinity sera égal à NaN
- Infinity divisé par un nombre sera égal à Infinity

Exercices

Les variables

Exercices - Les variables

- Demandez à l'utilisateur son prénom via prompt et stockez-le dans une variable let.
  Affichez une alert de bienvenue avec le message « Bonjour, [prénom] ! ».

- Demandez ensuite son âge via prompt et stockez-le dans une const.
  Affichez dans la console le type de la valeur reçue avec typeof.

- Expliquez pourquoi la valeur de l'age n'est-ce pas un number ?

- Affichez dans une alert le résumé : « Tu t'appelles [prénom] et tu as [âge] ans. »

Les types de données

Number

Les types de données

Déclaration d'une variable de type « Number »

Le type « Number » permet de stocker un nombre réel.

Exemple de déclaration de variable

```
const val1 = 42;
const val2 = 4_357;
const val3 = 9.99;
const val4 = 2e3;  // → 2000
```

Pour la lisibilité, le symbole « underscore » peut être utilisé comme séparateur.

Liste des méthodes d'une variable de type Number : Doc MDN - Number

Convertir en nombre

Le JavaScript permet de convertir une variable de type « String » :

- Méthode de conversion :
  - parseInt(..., ...)
  - parseFloat(...)
- Constructeur du type « number »
  - Number(...) → Conversion plus strict, elle échoue si le format est invalide (ex: 42a).

En cas d'échec de la conversion, ces méthodes renvoient le résultat « NaN ».

Rappel : Pour tester que la valeur est « NaN », il faut utiliser la méthode « isNaN(...) »

Convertir en nombre - Entier

La méthode « parseInt » permet d'obtenir la valeur entière d'une chaîne de caractères.

Celle-ci prend en paramètres :

- ➔ La chaîne de caractères à convertir
- ➔ La base (par défaut : 10)

```
const textNb = '42';
const nb1 = parseInt(textNb);      // → 42
const nb2 = parseInt(textNb, 10); // → 42
const nb3 = parseInt(textNb, 16); // → 66
const nb4 = parseInt('9.999');     // → 9

const textError = 'Dix';
const nb5 = parseInt(textError);  // → NaN
```

Convertir en nombre - Réel

La méthode « parseFloat » permet d'obtenir la valeur d'un réel.

Celle-ci ne prend qu'un seul paramètre :

➔ La chaîne de caractères à convertir.

```
const textReal = '3.14159';
const nb6 = parseFloat(textReal);     // → 3.14159

const textBad = '1,2345';
const nb7 = parseFloat(textBad);      // → 1

const textError2 = 'Demo';
const nb8 = parseFloat(textError2);   // → NaN
```

Les opérateurs arithmétiques

Le langage JavaScript supporte les opérateurs arithmétiques suivants :

- Addition
- Soustraction
- Multiplication
- Division
- Modulo
- Puissance

Peuvent être combinés à une affectation

```
const nb1 = 14;
const nb2 = 3;

// Addition
const r1 = nb1 + nb2;    // → 17

// Soustraction
const r2 = nb1 - nb2;    // → 11

// Multiplication
const r3 = nb1 * nb2;    // → 42

// Division
const r4 = nb1 / nb2;    // → 4,66666...

// Modulo (Reste d'une division entière)
const r5 = nb1 % nb2;    // → 2

// Puissance
const r6 = nb1 ** nb2;   // → 2744

// Syntaxe raccouci avec l'opérateur d'affectation
let r7 = 28;
r7 += nb1;               // → r7 = r7 + nb1;
```

Les opérateurs d'incrément et de décrément

Le langage JavaScript permet de réaliser des actions d'incrémentation et de décrémentation à l'aide d'un raccourci d'écriture.

Deux possibilités :

- Post-Action :
  Après l'utilisation de la variable.
- Pré-Action :
  Avant l'utilisation de la variable.

```
// Post-Incrementation & Post-Decrementation
let nbA = 5, nbB = 3;
const r1 = nbA++;          // → r1: 5 / nbA: 6
const r2 = nbB--;          // → r2: 3 / nbB: 2

// Pré-Incrementation & Pré-Decrementation
let nbC = 5, nbD = 3;
const r3 = ++nbC;          // → r3: 6 / nbC: 6
const r4 = --nbD;          // → r4: 2 / nbD: 2

// Exemple
let nbE = 5, nbF = 3;
const r5 = (nbE-- * ++nbF) + nbE;   // → r5 ?
```

Les constantes de « Number »

Quelques constantes utiles du pseudo-objet « Number » :

```
// La plus petite et la grande valeur numérique positive
Number.MIN_VALUE;
Number.MAX_VALUE;

// Les valeurs entieres minimum et maximum représentable en JS
Number.MIN_SAFE_INTEGER;   // -(2⁵³ - 1)
Number.MAX_SAFE_INTEGER;   //  (2⁵³ - 1)

// Le plus petit intervalle possible entre deux valeurs numériques
Number.EPSILON;
```

Les méthodes de « Number »

Quelques méthodes utiles du pseudo-objet« Number » :

```
// Détermine si la valeur est un entier
const r1 = Number.isInteger(42_000_000_000_000_123);      // → true

// Déterminer si la valeur peut être correctement représentée comme un entier
const r2 = Number.isSafeInteger(42_000_000_000_000_123);  // → false


const v1 = 42.13;

// Détermine si la valeur est un nombre fini
const r3 = Number.isFinite(v1);       // → true

// Détermine si la valeur vaut NaN (Not a Number)
const r4 = Number.isNaN(v1);          // → false
```

Remarque : Contrairement aux méthodes globales, les méthodes “isFinite” et “isNaN” du pseudo-objet « Number » ne réalise pas de conversion avant de tester la valeur.

L'objet « Math »

L'objet « Math » est un élément natif du Javascript qui permet d'utiliser :

- Des constantes
- Des fonctions mathématiques

Quelques exemples :

```
// La valeur de PI (π)
const v1 = Math.PI;          // → 3.141592653589793

// Arrondir un nombre
const v2 = 4.2;

const a1 = Math.round(v2);   // → 4 (l'arrondi mathématique)
const a2 = Math.floor(v2);   // → 4 (l'entier inférieur ou égal)
const a3 = Math.ceil(v2);    // → 5 (l'entier supérieur ou égal)

// Générer un nombre pseudo-aléatoire
const v3 = Math.random();
```

BigInt

Les types de données

Déclaration d'une variable de type « BigInt »

Le type « BigInt » permet de stocker des nombres entiers trop grands qui ne peuvent pas être représentés correctement par le type « Number ».

Exemple de déclaration de variable

```
const bi1 = 42n;
const bi2 = 13_000_000_000_000_123n;

const bi3 = BigInt('9007199254740991');
const bi4 = BigInt('0x1FA5B');
```

Liste des méthodes d'une variable de type BigInt : Doc MDN - BigInt

Les opérateurs

Les « BigInt » supportent les opérateurs arithmétiques, d'incrément et de décrément.

```
const nb1 = 42_000_000_000_000_013n;
const nb2 = 2n;

const r1 = nb1 * nb2;    // → 84000000000000026n
```

Remarque : Il n'est pas autorisé d'utiliser les opérateurs arithmétiques avec un autre type de donnée. Les deux valeurs doivent être de type « BigInt ».

String

Les types de données

Les chaînes des caractères

Les chaînes des caractères sont stockées dans des variables de type « String ».
Le contenu d'une chaîne de caractères doit être déclaré entre des guillemets.

C'est un type de donnée immuable, c'est-à-dire qu'il n'est pas possible de modifier son contenu une fois que celle-ci a été créée.

Exemple de déclaration de variable :

```
const name1 = 'Della';
const name2 = 'Balthazar';
console.log(name1.length);
```

Possibilité d'utiliser les simples ou doubles quotes

La propriété « length » permet d'obtenir la longueur.

Liste des méthodes d'une variable de type String : Doc MDN - String

Concaténation

Pour concaténer des chaînes de caractères, il est possible d'utiliser trois méthodes :

```
const firstname = 'Zaza';
const lastname = 'Vanderquack';

// Opérateur « + »
const text1 = 'Bienvenue ' + firstname + ' ' + lastname + ' !';

// Méthode « .concat »
const text2 = 'Bienvenue '.concat(firstname).concat(' ').concat(lastname).concat(' !');
const text3 = ''.concat(firstname, ' ', lastname, ' !');

// Template String
const text4 = `Bienvenue ${firstname} ${lastname} !`;
```

Le « Template String » nécessite d'utiliser les « backtick » et non des guillemets !

Boolean

Les types de données

Déclaration d'une variable de type « Boolean »

Le type « Boolean » permet de stocker une variable booléenne.

Exemple de déclaration de variable

```
const boolVrai = true;
const boolFaux = false;
```

Liste des méthodes d'une variable de type Boolean : Doc MDN - Boolean

Conversion implicite en booléen

Le Javascript réalise une conversion implicite lorsqu'une valeur est évaluée dans un contexte booléen (par exemple : dans la condition d'une structure conditionnelle).

```
if(person) {
  // ...
}
```

Il existe donc deux types de valeur possible :

- Truthy : Les valeurs considérées comme vraies.
- Falsy : Les valeurs considérées comme fausses.

Les valeurs « Truthy »

Voici un tableau d'exemple de valeur « Truthy » de JavaScript

Valeur	Type	Description
true	Boolean	Le mot clef « true »
42 / -3.14 / 13n	Number / BigInt	Des valeurs numériques différentes de 0
Infinity / -Infinity	Number	Une valeur représentant l'infinie
{ }	Object	Un objet
[ ]	Array	Un tableau
new Date()	Object	Une instance du prototype « Date »
"Della" / "false"	String	Une chaîne de caractères

Les valeurs « Falsy »

Voici un tableau avec la liste complète des valeurs « Falsy » de JavaScript

Valeur	Type	Description
null	Null	L'absence de valeur
undefined	Undefined	La valeur primitive « undefined »
false	Boolean	Le mot clef « false »
NaN	Number	Une valeur numérique invalide
0 / -0 / 0n	Number / BigInt	La valeur numérique zéro (positive ou négative)
"" / '' / ``	String	Une chaîne de caractères vide
document.all	Object	Le seul objet « Falsy » en JS → Pour des raisons de compatibilité

Object

Les types de données

Création d'un objet en JavaScript

Le type « Object » permet de stocker un ensemble de valeur sous forme de propriété.

Exemple de déclaration d'objet

```
// La méthode « create »
const o1 = Object.create();

// Le constructeur
const o2 = new Object();

// L'initialisateur d'objet
const o3 = {};

const person = {
  firstname: 'Della',
  lastname: 'Duck'
};
```

Interagir avec les propriétés d'un objet

Les « Property accessors » (Accesseurs de propriétés) permettent d'accéder aux différentes propriétés d'un objet à l'aide d'une notation avec un point ou des crochets.

```
const person = {
  firstname: 'Test',
  lastname: null
};

// Ecriture
person['firstname'] = 'Zaza';
person.lastname = 'Vanderquack';

// Lecture
const fullname = person.firstname + ' ' + person['lastname'];
```

L'opérateur « Optional chaining »

Lorsqu'on lit la valeur d'une propriété d'un objet « null » ou « undefined », une erreur est levée par le JavaScript.

```
> let person = null;
  person.firstname;
✕ ▶ Uncaught TypeError: Cannot read properties of null (reading 'firstname')
```

Pour éviter cette erreur, il est possible d'utiliser l'opérateur « Optional chaining ».

L'opérateur fonctionne comme la notation « . », sauf qu'il ne déclenche pas d'erreur sur les objets « null » ou « undefined », à la place, la valeur « undefined » est renvoyé.

```
> let person = null;
  person?.firstname;
< undefined
```

Exercices

Les types de données

Exercices - Les types de données

- Demandez un prix à l'utilisateur via prompt. Convertissez-le en réel, affichez-le dans la console, puis formatez-le pour un affichage en euros.

- Demandez le prénom et le nom de famille via deux prompt. Concaténez-les en une seule chaîne en utilisant une des méthodes vues au cours et affichez le résultat.

- Créez un objet personne avec nom, prénom, âge et ville.
  - Ajoutez une propriété email dynamiquement.
  - Affichez chaque propriété dans la console.

Interagir avec le DOM
- Les Bases -

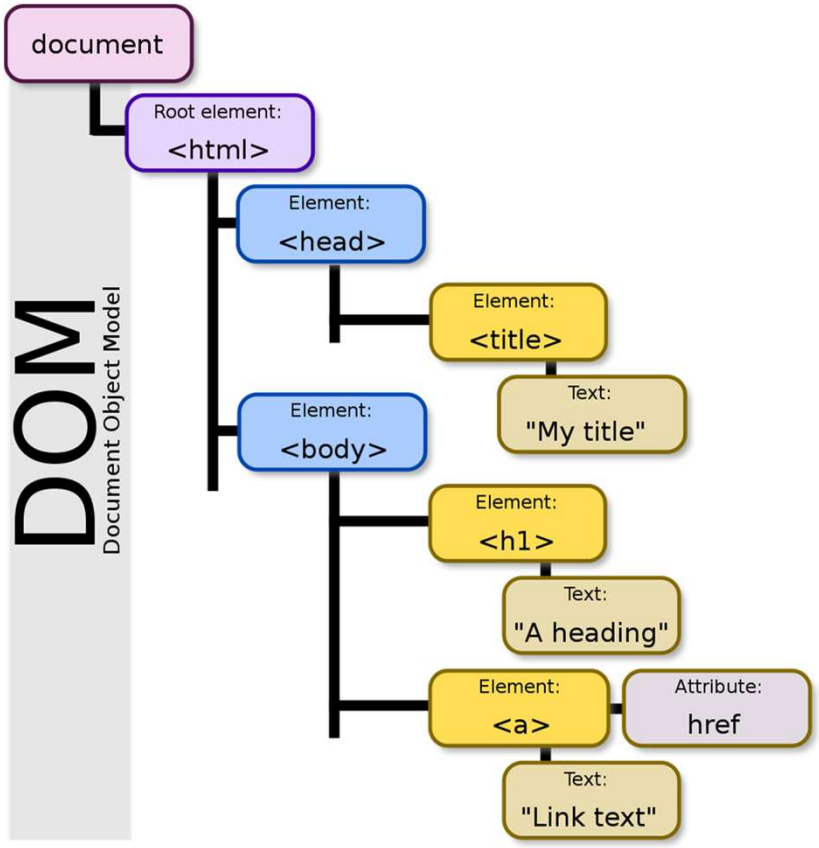
Introduction au DOM

Interagir avec le DOM - Les Bases

C’est quoi le DOM ?

Le DOM (Document object Model) est la représentation sous la forme d’objet des données qui composent la structure et le contenu d’une page Web et de XML.

Le DOM n’est pas un langage de programmation, ce n’est qu’une API qui met à disposition des méthodes pour permettre de réaliser des interactions avec des pages Web et du XML.



Apprentissage progressif du DOM

Bien que le DOM ne soit pas une matière qui fait partie du langage JavaScript, interagir avec celui-ci à travers l'utilisation de l'API de DOM est essentiel.

Dans le cadre du cours, nous aborderons progressivement l'utilisation de l'api du DOM.

Les différents points de matière vu durant ce cours seront :

- Manipulation basique avec une page
- Modifier les éléments d'une page
- Interagir avec des événements
- Gestion de formulaire en JavaScript

Manipulation basique avec une page

Interagir avec le DOM - Les Bases

Récupérer un élément à l'aide de son Id

La méthode « getElementById(...) » retourne l'élément ayant l'attribut id spécifié

- HTML

```html
<p id="display-message">Une balise d'un paragraphe</p>
```

- JavaScript

```javascript
const displayMessage = document.getElementById('display-message');
console.log(displayMessage);
```

Récupérer un élément à l'aide d'un sélecteur CSS

La méthode « querySelector(...) » retourne le premier élément qui correspond au sélecteur CSS

- HTML

```html
<div class="demo">
  <p>Element 1</p>
  <p class="target">Element 2</p>
  <p>Element 3</p>
</div>
```

- JavaScript

```javascript
const targetElement = document.querySelector('.demo > .target');
console.log(targetElement);
```

Récupérer des éléments à l'aide d'un sélecteur CSS

La méthode « querySelectorAll(...) » retourne une « NodeList » avec tous les éléments qui correspondent au sélecteur CSS.

- HTML

```html
<ul class="people">
  <li>Della Duck</li>
  <li>Zaza Vanderquack</li>
  <li>Balthazar Picsou</li>
</ul>
```

- JavaScript

```javascript
const people = document.querySelectorAll('ul.people > li:nth-child(odd)');
console.log(people);
```

Quelques propriétés des éléments du DOM

- .id : Obtenir ou modifier la valeur de l'attribut « id ».
- .innerHTML : Obtenir ou modifier le contenu html.
- .textContent : Obtenir ou modifier le contenu texte (Celui-ci est échappé).
- .className : Obtenir ou modifier la valeur de l'attribut « class ».
- .classList : Interaction avec la valeur de l'attribut « class » via les méthodes :
    - add(...) : Ajoute les classes spécifiées
    - remove(...) : Supprime les classes spécifiées
    - contains(...) : Vérifie la présence de la classe spécifiée
    - replace(..., ...) : Replace une classe par la nouvelle classe
    - toggle(...) : Ajoute ou supprime la classe spécifiée

Quelques propriétés des éléments du DOM

- .firstElementChild : Obtenir le premier élément de la balise.
- .lastElementChild : Obtenir le dernier élément de la balise.
- .children : Obtenir une collection avec tous les éléments enfants de la balise.

Remarque : Les propriétés ci-dessus permettent uniquement d'obtenir les nœuds de type « Element » de la balise ciblé. Les nœuds de type « Text » et « Commentaire » ne seront pas renvoyés.

Propriétés des balises de saisie de l'utilisateur

- .value : Obtenir ou modifier la valeur des « input », « textarea » et « select ».
- .valueAsNumber : Obtenir ou modifier la valeur de l'input en type « number ».
  Propriété disponible sur les inputs « number » et « date ».
- .valueAsDate : Obtenir ou modifier la valeur de l'input en type « Date ».
  Propriété disponible uniquement sur les inputs de type « date ».
- .checked : Obtenir ou modifier l'état une balise « input » de type « checkbox ».
- .disabled : Obtenir ou modifier l'état de désactivation de la balise.
  Lorsqu'une balise est désactivé, elle n'est plus cliquable ou focusable.
- .readOnly : Obtenir ou modifier l'état « lecture seule» de la balise.
  Propriété disponible sur les balises de saisie de contenant du

Interagir à l'événement « click » d'un élément

La méthode « .addEventListener('click', ...) » permet d'ajouter un code "en sommeil" sur l'événement clique de l'élément ciblé.

- HTML

```html
<input id="input-name" type="text">
<button id="btn-demo">Click here</button>
```

- JavaScript

```js
const inputName = document.getElementById('input-name');
const btnDemo = document.getElementById('btn-demo');

btnDemo.addEventListener('click', function() {
  const name = inputName.value;
  console.log('Bonjour ' + name);
});
```

Remarque : Les autres événements seront abordés ultérieurement dans le support.

Exercices

Interagir avec le DOM - Les Bases

Exercices - Interaction avec le DOM 1

Créez une page HTML contenant un titre <h1>, un paragraphe <p>, deux boutons et une liste <ul> avec quelques <li>.

- Récupérez le titre à l'aide de son id et le paragraphe à l'aide d'un sélecteur CSS. Modifiez leur contenu textuel depuis le JavaScript.
- Au clic du premier bouton, ajoutez une classe CSS au paragraphe (ex : une couleur) et affichez dans la console le nom de la classe active.
- Au clic du second bouton, ajoutez une autre classe CSS au paragraphe (ex : une font). Si la classe est déjà présente, elle doit être retirer.
- Bonus : Au clic d'un <li>, modifiez son texte pour indiquer qu'il a été sélectionné, et retirez cette indication si on clique de nouveau dessus.

Les opérateurs de condition

Les opérateurs d'égalité

Les opérateurs de condition

Les différents opérateurs d'égalité et d'inégalité

Le langage Javascript possède deux types opérateurs : faible et strict.

- Égalité (==) / Inégalité (!=)

L'opérateur tente de convertir les valeurs si celle-ci sont de type différents. Ensuite, il compare les valeurs et renvoie le résultat booléen.

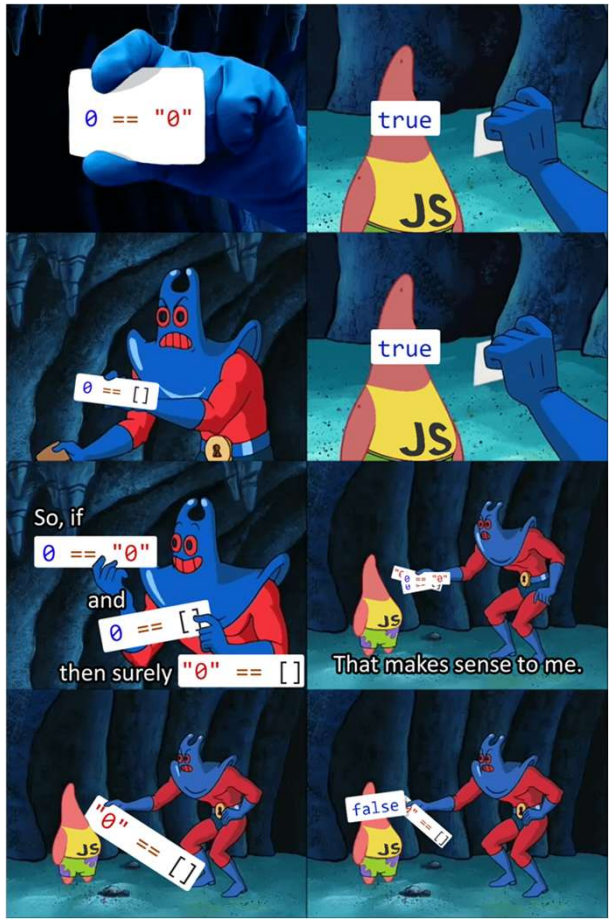
- Égalité stricte (===) / Inégalité stricte (!==)

L'opérateur strict considère comme fausse une égalité entre deux valeurs de type différents. Si le type est identique, il compare les valeurs et retourne le résultat.

Danger de l'utilisation des opérateurs « faibles »

L'utilisation des opérateurs d'égalités faibles est déconseillée car il est plus compliqué de prévoir son résultat dû au mécanisme de conversion réalisé.

Il est donc conseillé de privilégier l'utilisation des opérateurs stricts.



Opérateurs de comparaison

Les opérateurs de condition

Les différents opérateurs de comparaison

Le langage Javascript possède quatre opérateurs de comparaison :

- < : Plus petit
- <= : Plus petit et égal
- > : Plus grand
- >= : Plus grand et égal

Les types comparés par l'opérateur

Les opérateurs de comparaison permettent de tester :

- Deux valeurs de type numérique (Number ou BigInt)
- Deux chaînes de caractères (String)

Si les valeurs comparées sont d'un autre type, le moteur Javascript tentera de convertir les valeurs non-numériques et ensuite de réaliser une comparaison numérique.

NB : Si une des valeurs vaut « NaN », la comparaison sera toujours fausse.

Les opérateurs logiques

Les opérateurs de condition

Les opérateurs logiques

Le langage Javascript possède 3 opérateurs logiques :

- && : Le ET logique

Renvoie « true » si les deux opérandes sont « true ».

- || : Le OU logique

Renvoie « true » si au moins une des deux opérandes est « true ».

- ! : Le NON logique

Renvoie la valeur booléenne inversée de son opérande.

Exercices

Les opérateurs de condition

Exercices - Les opérateurs de condition

Pour chaque expression, donnez le résultat (Vrai ou Faux) sans exécuter le code.

- 5 > 3
- null === undefined
- 'b' > 'a'
- 0 === false
- 'JavaScript'.length > 5
- '42' == 42
- 'JavaScript'.startsWith('java')
- '10' > 9
- 'hello world'.indexOf('world') === 6
- 'Della Duck !'.length === 10
- Math.round(4.4) === Math.floor(4.9)
- 1 + 2 + '3' === '33'
- null == 0
- 'Riri Fifi Loulou'.split(' ').length === 3
- 4 + '5' + 6 === '96'
- (3.14159).toFixed(2) === 3.14

Traitement conditionnel

La structure « if...else »

Traitement conditionnel

Le « if...else »

La structure « if...else » permet d'exécuter du code en fonction de condition(s).

Elle peut être constituée de 3 blocs de code :

- if : Condition initiale à tester [Obligatoire]
- else if : Condition à tester si les précédentes sont fausses
- else : Si aucune condition n'a été validée

Syntaxe :

```
if (condition) {
  instruction1
}
```

```
if (condition) {
  instruction1
} else {
  instruction2
}
```

Exemple de structure « if...else »

```
btnDemo.addEventListener('click', function() {

  const nb = document.getElementById('input-nb').valueAsNumber;
  const result = document.getElementById('result-msg');

  if(isNaN(nb)) {
    result.innerText = 'La valeur est invalide !';
  }
  else if (nb === 42){
    result.innerText = 'Oh ! C\'est la réponse ultime !';
  }
  else {
    result.innerText = 'Merci pour votre participation...';
  }

});
```

La structure « switch »

Traitement conditionnel

Le « switch »

La structure « switch » permet d'exécuter du code en fonction d'une égalité stricte

Syntaxe :

```
switch (expression) {
  case valeurA:
    instructions1;
    break;
  case valeurB:
  case valeurC:
    instruction2;
    break;
  default:
    instructions_def;
    break;
}
```

Exemple de structure « switch »

```
switch (expr) {
  case 'Pommes':
    console.log('Pommes : 0.32 € le kilo.');
    break;
  case 'Cerises':
    console.log('Cerises : 3.00 € le kilo.');
    break;
  case 'Mangues':
  case 'Papayes':
    console.log('Mangues et papayes : 2.79 € le kilo.');
    break;
  default:
    console.log('Désolé, nous n\'avons plus de ' + expr + '.');
}
```

L’opérateur « ternaire »

Traitement conditionnel

L'opérateur ternaire

Cet opérateur permet d'affecter une variable sur base d'une condition.
Il est souvent utilisé comme raccourci pour la déclaration d'un « if...else ».

Sa syntaxe est composée de 3 opérantes :

- Une condition suivie d'un point d'interrogation (?)
- L'expression à réaliser si la condition est « Truthy », suivie de deux-point (:)
- Finalement, l'expression à réaliser si la condition est « Falsy »

```
const result = condition ? exprIfTrue : exprIfFalse;
```

Exemple d'utilisation de l'opérateur ternaire

```
// Ternaire simple
const r1 = isValid ? 'Validé !' : 'En attente de validation...';

// Ternaire sur un objet nullable
const r2 = person ? person.firstname : 'Anonyme';

// Ternaire enchaînée
const r3 = (choice === 1) ? 'Pommes'
  : (choice === 2) ? 'Cerises'
  : (choice === 3) ? 'Mangues'
  : 'Papayes';
```

L’opérateur « nullish coalescing »

Traitement conditionnel

L'opérateur « nullish coalescing »

L'opérateur permet de réaliser une condition avec la valeur « null » ou « undefined ».

Sa syntaxe se compose de 2 expressions séparées par deux points d'interrogation (??)

```
const result = student.yearResult ?? 5;
```

Il peut par exemple être utilisé pour :

- Affecter une valeur par défaut, si une variable ciblée n'a pas de valeur.
- Créer un court-circuit dans l'évaluation de l'instruction.

Remarque : contrairement à l'opérateur logique OU, cette opérateur ne base pas son résultat sur les valeurs « falsy ». Il ne prend en compte que « null » ou « undefined »

Exercices

Traitement conditionnel

Exercices - Traitement conditionnel

Créez une page HTML avec un champ de saisie et un bouton :

- Un <input type="number"> pour saisir une note sur 20. Si l'utilisateur n'a rien encodé, stockez « null » dans la variable.
- Au clic du bouton, affichez la note et son appréciation selon les seuils suivants :
  - ≥ 18 → "Excellent"
  - ≥ 14 → "Bien"
  - ≥ 10 → "Suffisant"
  - < 10 → "Insuffisant"

  Une note « null » n'obtient pas d'appréciation, afficher un message pour ce cas.

Exercices

Récapitulatif

Exercices - Récapitulatif (1/2)

Réalisez une calculatrice qui réutilise les notions vues jusqu'ici : variables, conversions, DOM et traitement conditionnel.

- Créez une page HTML contenant deux <input type="number">, une liste <select> proposant les opérations +, −, × et ÷, un bouton « Calculer » et un paragraphe pour le résultat.
- Au clic du bouton, récupérez les deux valeurs saisies et l'opération choisie.
- Si l'un des deux champs ne contient pas un nombre valide, affichez « Veuillez saisir deux nombres. » dans le paragraphe.
- Sinon, calculez le résultat à l'aide d'un « switch » sur l'opération et affichez-le sous la forme « 12 × 3 = 36 ».

Exercices - Récapitulatif (2/2)

- Pour une division par zéro, affichez « Division par zéro impossible. »
- Ajoutez la classe CSS « erreur » au paragraphe en cas d'erreur, et retirez-la quand le calcul réussit.
- Testez votre calculatrice avec ces cas :
  - 7 + 5 ⟶ « 7 + 5 = 12 »
  - 10 ÷ 0 ⟶ « Division par zéro impossible. »
  - un champ vide ⟶ « Veuillez saisir deux nombres. »
- Expliquez pourquoi 0.1 + 0.2 ne donne pas exactement 0.3.
- Pour aller plus loin : arrondissez à deux décimales avec « toFixed » et ajoutez un bouton « Effacer ».

Les Dates

L’objet de type « Date »

Les Dates

Les objets de type « Date »

Le type « Date » permet de représenter une date/heure en JavaScript.

Une date est stockée sous la forme d'un entier qui correspond au nombre de millisecondes écoulées depuis le premier janvier 1970 (UTC).

Pour initialiser une variable date, il est possible d'utiliser le constructeur :

- Sans paramètre (date et heure courante).
- Avec des valeurs individuelles pour la date et l'heure.
- Avec un nombre qui représente une valeur temporelle (en millisecondes) .
- Avec une chaîne de caractère au format ISO 8601 : YYYY-MM-DDTHH:mm:ss.sssZ

Les objets de type « Date »

Exemple de déclaration de variable de type « Date »

```
// Constructeur sans paramètre
const now = new Date();

// Valeurs individuelles pour la date et l'heure
// ! La valeur des mois commence à partir de 0
const d1 = new Date(2022, 5);                    // → 01/06/2022 00:00:00
const d2 = new Date(2014, 8, 9);                 // → 09/09/2014 00:00:00
const d3 = new Date(1995, 11, 4, 9, 42, 0);      // → 04/12/1995 09:42:00

// Nombre qui représente une valeur temporelle
const d4 = new Date(366485442000);               // → 12/08/1981 19:30:42

// Chaîne de caractère au format ISO 8601
const d5 = new Date("1995-12-17T03:24:00");      // → 17/12/1995 03:24:00
```

Méthodes d'un objet « Date »

Les Dates

Lire et modifier une date

Propriété	Getter	Setter
Jour du mois	.getDate()	.setDate(n)
Mois	.getMonth()	.setMonth(n)
Année	.getFullYear()	.setFullYear(n)
Heure	.getHours()	.setHours(n)
Minutes	.getMinutes()	.setMinutes(n)
Secondes	.getSeconds()	.setSeconds(n)

Remarque : La méthode « getMonth » retourne un index de 0 à 11 (janvier = 0, ...).

Obtenir un timestamp

La méthode « getTime » retourne le nombre de millisecondes écoulées depuis le 1er janvier 1970 (UTC). C'est ce qu'on appelle un timestamp.

Il est peut par exemple être utilisé pour comparer deux dates ou calculer une durée.

```
const debut = new Date('2025-01-01');
const fin   = new Date('2025-06-01');

const diffMs   = fin.getTime() - debut.getTime();
const diffJour = Math.round(diffMs / (1000 * 60 * 60 * 24));

console.log(diffJour); // 151
```

Afficher une date

Trois méthodes convertissent une date en chaîne : « toLocaleDateString », « toLocaleTimeString » et « toISOString ».

Les deux premières suivent la langue et le fuseau horaire du navigateur ; « toISOString » renvoie toujours l'heure UTC.

Remarque : une date ISO sans heure est lue en UTC. En Belgique en été (UTC+2), elle s'affiche donc à 02:00.

```
const date = new Date('2015-07-01');

date.toLocaleDateString('fr-BE');
// "01/07/2015"
date.toLocaleDateString('fr-BE', {
  day: 'numeric',
  month: 'long',
  year: 'numeric'
});
// "1 juillet 2015"
date.toLocaleTimeString('fr-BE');
// "02:00:00"
date.toISOString();
// "2015-07-01T00:00:00.000Z"
```

Exercices

Les dates

Exercices - Les Dates

Créez une page HTML avec plusieurs zones d'affichage.

- Déclarez dans le JavaScript la date du jour et une date de naissance.
- Calculez et affichez l'âge en années dans la page.
- Affichez la date du prochain anniversaire dans un format lisible où le mois apparaît en toutes lettres (ex : 25 juin 2025).
- Bonus : Calculez et affichez le nombre de jours restants avant ce prochain anniversaire.

NB : Les résultats doivent apparaître au chargement de la page.

Interaction avec le DOM
- Modifier la page -

Créer un nouvel élément

Interaction avec le DOM - Modifier la page

Créer un élément HTML

La méthode « createElement » de « document » permet de générer un nouvel élément. Elle prend en paramètre le type de balise à créer.

```
const della = document.createElement('p');

della.textContent = 'Della Duck';
della.id = 'della-42'
della.classList.add('person');

console.log(della);
// <p id="della-42" class="person">Della Duck</p>
```

Remarque : L'élément sera créé en mémoire, mais il ne sera pas ajouté à la page Web.

Les méthodes d'interaction

Interaction avec le DOM - Modifier la page

Les méthodes d'interaction

Les éléments HTML héritent des classes « Node » et « Element », celles-ci possèdent plusieurs méthodes qui permettent d'interagir avec la page Web.



Ces méthodes permettent de :

- Ajouter un élément enfant.
- Insérer un élément adjacent.
- Supprimer un élément.

Remarque : Un élément déjà présent dans la page sera déplacé, lorsqu'il est re-ajouté.

Ajouter un élément enfant

- parentElement.appendChild(element)

Permet d'ajouter un élément en tant que dernier enfant de la balise parent.

```
const parentElement = document.getElementById('people');
parentElement.appendChild(della);
```

- parentElement.insertBefore(element, referenceElement)

Permet d'ajouter un élément dans la balise parent avant un élément de référence.

```
const parentElement = document.getElementById('people');
parentElement.insertBefore(donald, della);
```

Ajouter un élément enfant

- parentElement.append(element1, ..., elementN)

Permet d'ajouter des éléments en tant que derniers enfants de la balise parent.

```
const parentElement = document.getElementById('people');
parentElement.append(balthazar, zaza);
parentElement.append(archibald);
```

- parentElement.prepend(element1, ..., elementN)

Permet d'ajouter des éléments en tant que premiers enfants de la balise parent.

```
const parentElement = document.getElementById('people');
parentElement.prepend(flagada, daisy);
parentElement.prepend(missTick);
```

Insérer un élément adjacent

- referenceElement.insertAdjacentElement(position, element)

Permet d'insérer un élément à une position par rapport à l'élément de référence.
La position où celui-ci est ajouté dépend de la valeur du paramètre :

- beforebegin : Avant l'élément de référence.
- afterbegin : En premier enfant à l'intérieur de l'élément de référence.
- beforeend : En dernier enfant à l'intérieur de l'élément de référence.
- afterend : Après l'élément de référence.

```
const parentElement = document.getElementById('people');
daisy.insertAdjacentElement('afterend', gontran);
```

Insérer un élément adjacent

- referenceElement.before(element1, ..., elementN)

Permet d'insérer des éléments avant l'élément référence.

```
balthazar.before(riri, fifi);
```

- referenceElement.after(element1, ..., elementN)

Permet d'insérer des éléments après l'élément référence.

```
balthazar.after(loulou, violette);
```

Supprimer un élément

- parentElement.removeChild(targetElement)

Permet de supprimer l'élément ciblé de la balise parent.

```
const parentElement = document.getElementById('people');
parentElement.removeChild(archibald);
```

- targetElement.remove()

Permet de supprimer l'élément ciblé.

```
missTick.remove();
```

Exercices

Interaction avec le DOM - Modifier la page

Exercices - Interaction avec le DOM 2

Créez une page HTML avec un <input id="input-fruit" type="text"> pour saisir le nom d'un fruit, un bouton "Ajouter" et une liste vide <ul id="ma-liste">.

- Au clic du bouton, récupérez la valeur de l'input :
  - Si le champ est vide, affichez un message d'erreur dans la page ou via alert.
  - Si le champ est rempli, ajoutez dynamiquement un <li> et videz l'input.
- Ajoutez un bouton "Tout supprimer" qui vide la liste entière.
- Bonus : Ajoutez un bouton "Supprimer" sur chaque <li> qui retire l'élément.

Traitement itératif

La structure « while »

Traitement itératif

Le « while »

La structure « while » permet de répéter l'exécution d'un bloc de code.

Fonctionnement de la boucle :

- Test de la condition.
- Si la condition est « vraie », exécution du code
- Retour à la condition

Syntaxe :

```
while (condition) {
  instruction
}
```

Exemple de structure « while »

```
btnDemo.addEventListener('click', function() {

  const nb = document.getElementById('input-nb').valueAsNumber;
  let count = 0;

  while (count < nb) {

    const elem = document.createElement('p');
    elem.textContent = count;
    zone.append(elem);

    count++;

  }

});
```

La structure « do...while »

Traitement itératif

Le « do...while »

La structure « do...while » permet d'exécuter un bloc de code et de le répéter.

Fonctionnement de la boucle :

- Exécution du code
- Test de la condition.
- Si la condition est « vraie », retour à l'exécution du code

Syntaxe :

```
do {
  instruction
} while (condition);
```

Exemple de structure « do...while »

```
btnDemo.addEventListener('click', function() {

  const nb = document.getElementById('input-nb').valueAsNumber;
  let count = 0;

  do {

    const elem = document.createElement('p');
    elem.textContent = count;
    zone.append(elem);

    count++;

  } while (count < nb);

});
```

La structure « for »

Traitement itératif

Le « for »

La structure « for » est une boucle qui possède un bloc de code d'initialisation et final.

Fonctionnement de la boucle :

- Exécution du code d'initialisation
- Test de la condition.
- Si la condition est « vraie », exécution du code de la boucle et du code final.
- Retour à la condition

Syntaxe :

```
for (initialisation; condition; final) {
  instruction
}
```

Exemple de structure « for »

```
btnDemo.addEventListener('click', function() {

  const nb = document.getElementById('input-nb').valueAsNumber;

  for (let count = 0; count < nb; count++) {

    const elem = document.createElement('p');
    elem.textContent = count;
    zone.append(elem);

  }

});
```

Instructions « continue » et « break »

Traitement itératif

Les instructions d'une structure itérative

Les mots clefs « continue » et « break » sont deux instructions pouvant être utilisées dans une structure itérative.

- continue

Permet d'arrêter le traitement actuel et de passer à l'itération suivante de la boucle.

- break

Permet de terminer la boucle et de passer au code suivant.

Exercices

Traitement itératif

Exercices - Traitement itératif

Créez une page avec un bouton « Générer » et un conteneur .

- Au clic du bouton, utilisez une boucle for pour générer 25 cases (des stylisés en carré) numérotées de 1 à 25 et les insérer dans la grille.
- Utilisez une boucle while pour parcourir les cases générées et colorez en rouge les cases dont le numéro est multiple de 3, en vert les autres.
- Ajoutez un bouton « Effacer » qui supprime toutes les cases de la grille.

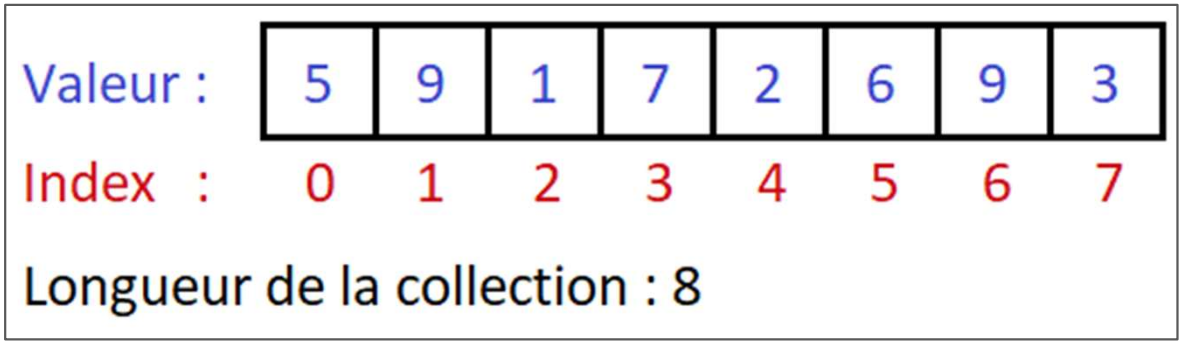
Les collections indexées

Déclaration et utilisation

Les collections indexées

C’est quoi une collection indexée ?

Une collection indexée est une structure de données qui permet de stocker un ensemble de valeurs dans une variable et de les rendre accessibles à l’aide d’un index.



La valeur d’index commence à 0 et celui-ci est une valeur entière positive.

Les collections indexées en JavaScript

En JavaScript, les collections indexées sont implémentées par l'objet global « Array ».

Particularités d'une collection de type « Array » :

- On peut accéder à leur contenu à l'aide de l'opérateur d'accès [ ].
- Le nombre d'éléments est dynamique.
- Les valeurs stockées peuvent être de type différents (non conseillé).
- Les structures itératives permettent de parcourir leur contenu.
- Elles possèdent des méthodes.

Les objets de type « Array »

Exemple de déclaration de variable de type « Array »

```
// Déclaration à l'aide de la notation littérale :
const emptyTab1 = [];
const drake = ['Ludwig', 'Klara', 'Corvus', 'Anya'];

// Déclaration à l'aide du constructeur :
// - Sans paramètre
const emptyTab2 = new Array();

// - Un seul paramètre de type number (de 0 à 2^32-1)
//   Créer un tableau avec X éléments vides
const zone = new Array(5);

// - Plusieurs paramètres
const vals = new Array(5, 3, 1);
const rapetou = new Array('La Science', 'La Gonflette', 'Burger');
```

Les objets de type « Array »

L'opérateur d'accès [ ] permet d'obtenir ou de modifier la valeur d'un élément du tableau à l'aide de son index.

```
const people = ['Della', 'Balthazar', 'Zaza', 'Gontran', 'Daisy'];

// Obtenir la valeur de l'élément à l'index 1
console.log(people[1]);   // → Balthazar

// Modifier l'élément à l'index 3
people[3] = 'Donald';

// Ajouter un nouvel élément (index cible > index max)
people[6] = 'Riri'
```

Si on affecte une valeur à un index supérieur à l'index maximum, l'élément est ajouté.

Les objets de type « Array »

Les méthodes « console.log(...) » et « console.table(...) » permettent de visualiser facilement le contenu du tableau dans le terminal

```
console.log(people);
▶ (7) ['Della', 'Balthazar', 'Zaza', 'Donald', 'Daisy', empty, 'Riri']
```

```
console.table(people)
```

(index)	Value
0	'Della'
1	'Balthazar'
2	'Zaza'
3	'Donald'
4	'Daisy'
6	'Riri'

```
▶ Array(7)
```

Les objets de type « Array »

La propriété « length » permet d'obtenir la longueur du tableau.

```
// Trois éléments
const tab1 = ['Riri', 'Fifi', 'Loulou'];
console.log(tab1.length);   // → 3

// Cinq éléments vides
const tab2 = new Array(5);
console.log(tab2.length);   // → 5

// Trois éléments et six éléments vides
const tab3 = ['Archiblad', 'Gabby'];
tab3[8] = 'Miss Tick';
console.log(tab3.length);   // → 9
```

Remarque : La propriété prend en compte les éléments vides du tableau.

Les méthodes des collections

Les collections indexées

Ajouter des éléments

- .push(element1, ..., elementN)

Permet d'ajouter un ou plusieurs éléments à la fin de la collection.

```
const personnages = ['Balthazar', 'Zaza'];
personnages.push('Donald', 'Daisy');    // → ['Balthazar', 'Zaza', 'Donald', 'Daisy']
personnages.push('Della');              // → ['Balthazar', 'Zaza', 'Donald', 'Daisy', 'Della']
```

- .unshift(element1, ..., elementN)

Permet d'ajouter un ou plusieurs éléments au début de la collection

```
const personnages = ['Balthazar', 'Zaza'];
personnages.unshift('Donald', 'Daisy'); // → ['Donald', 'Daisy', 'Balthazar', 'Zaza']
personnages.unshift('Della');           // → ['Della', 'Donald', 'Daisy', 'Balthazar', 'Zaza']
```

Supprimer un élément

- .pop()

Permet d'obtenir et de supprimer le dernier élément de la collection

```
const personnages = ['Balthazar', 'Zaza', 'Della', 'Miss Tick'];
const target = personnages.pop();
console.log(personnages);                // → ['Balthazar', 'Zaza', 'Della']
console.log(target);                     // → 'Miss Tick'
```

- .shift()

Permet d'obtenir et de supprimer le premier élément de la collection

```
const personnages = ['Balthazar', 'Zaza', 'Della', 'Miss Tick'];
const target = personnages.shift();
console.log(personnages);                // → ['Zaza', 'Della', 'Miss Tick']
console.log(target);                     // → 'Balthazar'
```

Ajouter et supprimer des éléments

- .splice(indexDebut, deleteCount [, element1, ..., elementN])

Permet de supprimer de 0 à plusieurs éléments à partir de l'index reçu et ensuite d'ajouter de 0 à plusieurs éléments sur la collection.

```
const personnages = ['Balthazar', 'Miss Tick', 'Archibald', 'Della'];
const removed = personnages.splice(1, 2, 'Riri', 'Fifi', 'Loulou');
console.log(personnages);       // → ['Balthazar', 'Riri', 'Fifi', 'Loulou', 'Della']
console.log(removed);           // → ['Miss Tick', 'Archibald']
```

- .toSpliced(indexDebut, deleteCount, element1, ..., elementN)

Traitement identique à la méthode « .splice(...) » sauf que celle-ci crée une copie de la collection qui est modifiée et renvoyée.

Convertir en chaîne de caractères

- .join([séparateur])

Renvoie une chaîne de caractères en concaténant tous les éléments d'un tableau séparés par le séparateur (par défaut : «,»)

```
const personnages = ['Balthazar', 'Zaza', 'Donald', 'Daisy', 'Della'];
const text1 = personnages.join();
console.log(text1);              // → 'Balthazar,Zaza,Donald,Daisy,Della'
const text2 = personnages.join(' - ');
console.log(text2);              // → 'Balthazar - Zaza - Riri - Fifi - Loulou - Della'
```

- .toString()

Renvoie une chaîne de caractères représentant le tableau spécifié et ses éléments. Cette méthode génère le même résultat que la méthode « .join() ».

Trier les éléments

- .sort([methodeComparaison])

Permet de trier les éléments d'un tableau sur base d'une méthode qui doit retourner un nombre positif, négatif ou zéro.

```
const vals = [42, 13, -1, 4];
vals.sort(function(a, b) {
  return a - b;
});
console.log(vals);
// → [ -1, 4, 13, 42 ]
```

```
const names = ['Balthazar', 'Léon', 'Édith', 'Lena', 'Zaza', 'Lily'];
names.sort(function(a, b) {
  return a.localeCompare(b);
});
console.log(names);
// → ['Balthazar', 'Édith', 'Lena', 'Léon', 'Lily', 'Zaza']
```

- .toSorted([methodeComparaison])

Traitement identique à la méthode « .sort(...) » sauf que celle-ci crée une copie de la collection qui est modifiée et renvoyée.

Inverser l'ordre des éléments

- .reverse()

Permet d'inverser l'ordre des éléments d'un tableau sur base d'une fonction qui doit retourner un nombre positif, négatif ou zéro.

```
const names = ['Balthazar', 'Léon', 'Édith', 'Lena', 'Zaza', 'Lily'];
names.reverse();
console.log(names);
// → ['Lily', 'Zaza', 'Lena', 'Édith', 'Léon', 'Balthazar']
```

- .toReversed()

Traitement identique à la méthode « .reverse(...) » sauf que celle-ci crée une copie de la collection qui est modifiée et renvoyée.

Fusionner des tableaux

- .concat(element1, ..., elementN)

Permet de générer un tableau en fusionnant deux ou plusieurs tableaux.

```
const picsou = ['Balthazar', 'Hortense'];
const duck = ['Della', 'Donald'];
const guest = ['Zaza', 'Mamie Baba'];
const familly1 = picsou.concat(duck).concat(guest);
const familly2 = [].concat(picsou, duck, guest);
// → ['Balthazar', 'Hortense', 'Della', 'Donald', 'Zaza', 'Mamie Baba']
```

Extraire des portions de tableaux

- .slice([start,[end]])

Permet de générer un tableau qui contient la copie d'une portion du tableau cible.
La portion sélectionnée dans le tableau est définie par des index.

```
const personnages = ['Donald', 'Balthazar', 'Zaza', 'Della', 'Riri', 'Fifi', 'Loulou'];
// Index de début (inclusif) et de fin (exclusif)
const grp1 = personnages.slice(1, 3);
console.log(grp1);  // → ['Balthazar', 'Zaza']
// Index de début uniquement
const grp2 = personnages.slice(3);
console.log(grp2);  // → ['Della', 'Riri', 'Fifi', 'Loulou']
// Copie complète du tableau (pas d'index)
const grp3 = personnages.slice();
console.log(grp3);  //  → ['Donald', 'Balthazar', 'Zaza', 'Della', 'Riri', 'Fifi', 'Loulou']
```

Rechercher un élément

- .includes(elementRecherché [, indiceDebut]);

Permet de déterminer si le tableau contient une valeur recherchée.

```
const vals = [3, -5, 22, 39, -3, 22, 42, 14, 0];
const test1 = vals.includes(42);       // → true
const test2 = vals.includes(-5, 3);    // → false
const test3 = vals.includes(14, -3);   // → true
```

- .some(methodeRecherche) / .every(methodeRecherche)

Vérifie si un élément (.some) ou si tous les éléments (.every) valident la condition.

```
const vals = [3, -5, 22, 39, -3, 22, 42, 14, 0];
const test4 = vals.some(function(nb) {
  return nb % 13 === 0;                // → true
});
```

Obtenir un élément

- .find(methodeRecherche) / .findLast(methodeRecherche)

Permet d'obtenir la valeur correspondant à une méthode de test.

```
const personnages = ['Donald', 'Balthazar', 'Zaza', 'Della', 'Riri', 'Fifi', 'Loulou'];
// Le premier personnage avec un prénom de moins de 5 lettres
const r1 = personnages.find(function(prenom) {
  return prenom.length < 5
});
console.log(r1);                          // → Zaza
// Le dernier personnage avec un prénom de moins de 5 lettres
const r2 = personnages.findLast(function(prenom) {
  return prenom.length < 5;
});
console.log(r2);                          // → Fifi
// Le premier personnage avec les lettres 'za' dans son prénom
const r3 = personnages.find(function(prenom) {
  return prenom.includes('za');
});
console.log(r3);                          // → Balthazar
```

Obtenir l'index d'un élément

- .findIndex(methodeRecherche) / .findLastIndex(methodeRecherche)

Permet d'obtenir la valeur de l'index correspondant à une méthode de test.

```
const personnages = ['Donald', 'Balthazar', 'Zaza', 'Della', 'Riri', 'Fifi', 'Loulou'];
// Le premier personnage avec un prénom de moins de 5 lettres
const r1 = personnages.findIndex(function(prenom) {
  return prenom.length < 5
});
console.log(r1);                          // → 2
// Le dernier personnage avec un prénom de moins de 5 lettres
const r2 = personnages.findLastIndex(function(prenom) {
  return prenom.length < 5;
});
console.log(r2);                          // → 5
// Le premier personnage avec les lettres 'za' dans son prénom
const r3 = personnages.findIndex(function(prenom) {
  return prenom.includes('za');
});
console.log(r3);                          // → 1
```

Appliquer un filtre à la collection

- .filter(methodeFiltre)

Permet d'obtenir la copie filtrée d'un tableau sur base d'une fonction.

```
const personnages = ['Donald', 'Balthazar', 'Zaza', 'Della', 'Riri', 'Fifi', 'Loulou'];
// Obtenir tous les prénoms avec la lettre 'a'
const r1 = personnages.filter(function(prenom) {
  return prenom.includes('a');
})
console.log(r1);        // → ['Donald', 'Balthazar', 'Zaza', 'Della']

const vals = [3, -5, 22, 39, -3, 22, 42, 14, 0];
// Obtenir tous les nombres pairs
const r2 = vals.filter(function(nb) {
  return nb % 2 === 0;
});
console.log(r2);        // → [22, 22, 42, 14, 0]
```

Transformer la collection

- .map(methodeTransformation)

Permet d'obtenir la copie modifiée d'un tableau sur base d'une fonction.

```
const personnages = ['Donald', 'Balthazar', 'Zaza', 'Della'];
// Obtenir tous les prénoms en majuscule
const r1 = personnages.map(function(prenom) {
  return prenom.toLocaleUpperCase('fr-be');
})
console.log(r1);        // → ['DONALD', 'BALTHAZAR', 'ZAZA', 'DELLA']

const vals = [3, -5, 22, 39, -3, 22, 42, 14, 0];
// Obtenir le triple de tous les nombres
const r2 = vals.map(function(nb) {
  return nb * 3;
});
console.log(r2);        // → [9, -15, 66, 117, -9, 66, 126, 42, 0]
```

Exercices

Les collections indexées

Exercices - Les collections indexées

Partez du tableau : [5, 12, 3, 8, 1, 2, 7]

- Ajoutez la valeur 15 à la fin et la valeur 9 au début.
- Triez le tableau par ordre croissant, puis inversez-le.
- Retirez le dernier élément et le premier élément.
- Vérifiez si le tableau contient la valeur 8
- Créer une copie du tableau qui contiendra le double des nombres du tableau original inférieur à 10.
- Trouvez la position du premier nombre entre 5 à 10 dans les deux tableaux.
- Affichez les tableaux en console avec « console.table() ».

Traitement itératif
- Les collections -

Les structures itératives

Traitement itératif - Les collections

Parcourir une collection avec une boucle

Pour parcourir les différents éléments d'une collection indexée, il est possible d'utiliser les structures itératives précédemment abordées :

- « while »
- « do...while »
- « for »

Pour cela, il sera souvent nécessaire de créer une variable qui nous permettra de définir la condition de fin de boucle, pour éviter de "sortir" de la collection.

Exemple avec la structure « while »

```
const personnages = ['Donald', 'Balthazar', 'Zaza', 'Della', 'Riri', 'Fifi', 'Loulou'];

// Variable avec la valeur du dernier index de la collection
let index = personnages.length - 1;

// Traitement itératif
while(index >= 0) {

  const perso = personnages[index];
  console.log(`> ${perso}`);

  index--;
}
```

Exemple avec la structure « for »

```
const personnages = ['Donald', 'Balthazar', 'Zaza', 'Della', 'Riri', 'Fifi', 'Loulou'];

// Traitement itératif
for(let index = 0; index < personnages.length; index++) {

  const perso = personnages[index];
  console.log(`> ${perso}`);

}
```

Les autres boucles pour les collections

Bien que ces structures itératives soient suffisantes, le langage JavaScript propose d'autres solutions pour manipuler les collections.

Celui-ci permet également d'utiliser d'autres boucles adaptées aux collections :

- La méthode « forEach(...) » du prototype Array
- La structure « for .. of »
- La structure « for .. in »

La méthode « Array.foreach »

Traitement itératif - Les collections

Le « forEach »

La méthode « forEach » du prototype « Array » permet d'exécuter un bloc de code pour chaque élément d'une collection à l'aide d'une méthode reçue en paramètre.

La méthode peut posséder jusqu'à 3 paramètres :

- Premier : La valeur de l'élément en cours de traitement.
- Deuxième (Optionnel) : L'indice de l'élément en cours de traitement.
- Troisième (Optionnel) : La collection utilisée par la méthode « forEach ».

Syntaxe :

```
collection.forEach(function (value, index, array) {
  instruction
});
```

Exemple de la méthode « forEach »

```
const personnages = ['Donald', 'Balthazar', 'Zaza', 'Della', 'Riri', 'Fifi', 'Loulou'];

// Traitement itératif
personnages.forEach(function (perso) {

  console.log(`> ${perso}`);

});
```

Limitation de la méthode « forEach »

La méthode « forEach » parcourt la totalité des éléments de la collection.

Il n'est pas possible d'arrêter le traitement d'une boucle « forEach », à part en déclenchant une exception (code d'erreur).

Si vous avez besoin d'arrêter la boucle, utiliser plutôt :

- Une boucle classique (for, while, do .. while)
- Une boucle for .. of
- Une boucle for .. in
- Une méthode des collections (filter, find, findIndex, ...)

La structure « for...of »

Traitement itératif - Les collections

Le « for .. of »

La structure « for .. of » permet de parcourir des objets itérables (Array, Map, String, ...).

Cette boucle permet de parcourir tous les éléments d'un itérable.
La variable initialisée par la structure reçoit une copie de la valeur parcourue.

Syntaxe :

```
for (const value of element) {
  instruction
}
```

Exemple de structure « for .. of »

```
const personnages = ['Donald', 'Balthazar', 'Zaza', 'Della', 'Riri', 'Fifi', 'Loulou'];

// Traitement itératif
for (const perso of personnages) {
  console.log(`> ${perso}`);
}
```

La structure « for...in »

Traitement itératif - Les collections

Le « for .. in »

La structure « for .. in » permet de parcourir les propriétés énumérables d'un objet.

Bien que cette boucle permet de parcourir les collections, il n'est pas recommandé de l'utiliser car :

- Elle ne permet pas de garantir l'ordre de parcours de la collection.
- Elle manipule les index (int) sous la forme de chaîne de caractères (string)

Syntaxe :

```
for (const key in element) {
  instruction
}
```

Exemple de structure « for .. in »

```
// - Exemple avec une collection
const personnages = ['Donald', 'Balthazar', 'Zaza', 'Della', 'Riri', 'Fifi', 'Loulou'];

// Traitement itératif
for (const key in personnages) {

  const perso = personnages[key];
  console.log(`> ${perso}`);

}

// - Exemple avec un objet
const students = { 'zava42': 'Zaza Vanderquack', 'ridu19': 'Riri Duck', 'cama01': 'Castor Major' };

// Traitement itératif
for (const key in students) {

  const student = students[key];
  console.log(`${key} -> ${student}`);

}
```

Exercices

Traitement itératif - Les collections

Exercices - Traitement itératif et collections

- Partez du tableau ['Balthazar', 'Della', 'Donald', 'Riri', 'Zaza']
  Utilisez la boucle « .forEach() » pour créer dynamiquement un <li> par personnage dans une liste <ul> de la page, en affichant sa position et son nom.
- Avec la boucle « for...in », parcourez l'objet suivant :
  { lastname: 'Duck', firstname: 'Balthazar', city: 'Canardville' }
  Pour chaque paire clef/valeur, créez dynamiquement un <p> dans la page.
- Expliquez pourquoi « for...in » est déconseillé pour parcourir les tableaux ?
- Bonus : Créez une collection d'objets avec des produits, chaque produit possède un nom, une description (optionnelle) et un prix.
  Ensuite, afficher les produits dans une liste <ul> en utilisant la boucle « for...of ».

Les fonctions

Déclaration et utilisation

Les fonctions

Déclaration d'une fonction

Les fonctions permettent de stocker un bloc de code et de l'exécuter.

Les fonctions ont comme objectif :

- D'éviter de répéter du code.
- Diviser le code en un ensemble de fonction simple.

La fonction renvoie la valeur retournée par l'instruction « return » ou « undefined » si la fin du code est atteint.

```
// Déclaration d'une fonction d'addition
function addTwoNumber(nb1, nb2) {
  // Traitement
  const result = nb1 + nb2;
  // Valeur retournée par la fonction
  return result;
}

// Déclaration d'une fonction de log
function logger(message) {
  // Traitement
  const now = new Date().toISOString();
  console.log(`[${now}] ${message}`);
}

// Utilisation des fonctions
const val = addTwoNumber(13, 29);
logger(`Le résultat vaut ${val}`);
```

Déclaration d'une fonction

Ils est également possible de déclarer une fonction en l'affectant à une variable.
Cela permet, par exemple, de garantir son unicité en l'affectant à une constante.

```
// Déclaration d'une fonction dans une variable
const numberIsEven = function (nb) {
  return nb % 2 === 0;
}

// Utilisation de la fonction
const test1 = numberIsEven(42);
console.log(`Est ce que 42 est pair : ${test1 ? 'oui' : 'non'}`);
```

Remarque : contrairement à la syntaxe classique, une fonction déclaré à l'aide d'une variable ne peut pas être utilisé avant sa déclaration.

Les paramètres

Les fonctions

Paramètres d'une fonction

Lors de la déclaration d'une fonction, il est possible de définir ses paramètres.

Les paramètres peuvent être :

- Une valeur primitive (number, ...)
- Un objet
- Une collection
- Une fonction

```
function getCartPrice(cart, tva) {
  let total = 0;
  for (const product in cart) {
    total += (product.price * product.quantity);
  }

  const tax = total * (tva / 100);
  return total + tax;
}

const cart = [
  { name: 'Pomme', quantity: 10, price: 0.32 },
  { name: 'Cerise', quantity: 5, price: 3.00 },
  { name: 'Mangue', quantity: 5, price: 2.79 },
];

const price = getCartPrice(cart, 6);
```

Paramètres avec une valeur par défaut

Il est possible de définir une valeur par défaut aux paramètres d'une fonction.

Pour cela, il faut définir la valeur par défaut lors de la déclaration du paramètre à l'aide de l'opérateur égal.

Cette valeur sera utilisée si :

- Aucune valeur n'a été définie
- La valeur « undefined » a été passée

```
function sayHello(firstname = 'John', lastname = 'Smith') {
  console.log(`Bonjour ${firstname} ${lastname} !`);
}

sayHello('Della');
// → Bonjour Della Smith !

sayHello('Zaza', 'Vanderquack');
// → Bonjour Zaza Vanderquack !

sayHello(undefined, 'Snow');
// → Bonjour John Snow !

sayHello();
// → Bonjour John Smith !
```

Paramètre avec l'opérateur « rest »

L'opérateur « rest » permet de définir un paramètre qui peut obtenir un nombre indéfini d'arguments.

Les valeurs reçues seront stockées sous la forme d'une collection (Array).

La syntaxe de l'opérateur est « ... »

```
function combine(separator = ' > ', ...names) {
  if(names.length === 0) {
    return 'No data !';
  }
  let result = names[0];
  for (let i = 1; i < names.length; i++) {
    result += separator + names[i];
  }
  return result;
}

const r1 = combine(', ', 'Della', 'Riri', 'Zaza');
// → "Della, Riri, Zaza"

const r2 = combine(undefined, 'Baltazar', 'Donald');
// → "Balthazar > Donald"
```

Les fonctions fléchées

Les fonctions

Déclaration d'une fonction fléchée

L'écriture fléchée permet de déclarer une fonction à l'aide de la syntaxe :

« (...) => { ... } ».

Note : lorsqu'il n'y a qu'un seul paramètre, les parenthèses sont optionnelles.

Lors de la déclaration d'une fonction fléchée, il est possible ne pas définir de corps à celle-ci.

Dans ce cas, il ne faut pas mettre les accolades et le « return » est implicite.

```
// Exemple de fonction fléchée
const example01 = () => {
  return 42;
}
const example02 = (x) => {
  return x + 5;
}
const example03 = (x, y) => {
  return x * y;
}

// Exemple de fonction "lambda"
const example04 = () => 42;
const example05 = (x) => x + 5;
const example06 = (x, y) => x * y;
```

Les « Callback »

Les fonctions

Principe de « Callback »

Les callback (fonction de rappel en francais) consistent à envoyer une fonction "interne" à une autre fonction en tant qu'argument.

Cette fonction "interne" sera ensuite invoquée par la fonction pour réaliser une action.

Exemple de fonction (qu'on a utilisé) qui attendent un callback en paramètre :

- elementDOM.addEventListener('click', callback)
- array.find(callback)
- array.filter(callback)

Exemple d'implémentation d'un « Callback »

```
// Déclaration d'une fonction avec un callback
function numbersValidator(array, conditionCallback) {
  const results = [];
  for(const elem of array) {
    const nb = parseFloat(elem);
    if(!isNaN(nb) && (!conditionCallback || conditionCallback(nb))) {
      results.push(nb);
    }
  }
  return results;
}

// Utilisation de la fonction avec un callback
const elements = [42, 13, 9, 6, 40, 22];
const evenNumbers = numbersValidator(elements, (nb) => nb % 2 === 0);
```

Exercices

Les fonctions

Les fonctions

- Écrivez une fonction multiplier(a, b) avec une valeur de 1 par défaut pour b. Testez-la avec deux arguments, puis avec un seul.
- Écrivez une fonction somme(...nombres) utilisant l'opérateur rest pour additionner un nombre indéfini d'arguments.
- Réécrivez multiplier en fonction fléchée avec retour implicite.
- Bonus : Écrivez une fonction récursive factorielle(n) qui calcule la factorielle d'un nombre entier positif (rappel : factorielle(5) = 5 × 4 × 3 × 2 × 1 = 120).

Exercices

Récapitulatif - Catalogue de produits

Exercices - Récapitulatif (1/4)

Réalisez un catalogue de produits qui réutilise les notions vues jusqu'ici : DOM, tableaux, boucles et fonctions.

- Créez une page HTML avec une liste <ul id="catalogue">, un paragraphe <p id="resume"> et une zone pour les boutons.
- Copiez le tableau « produits » ci-contre sans le modifier : les cas de test de la dernière slide en dépendent.

```
const produits = [
  { nom: 'Clavier', prix: 49.9, stock: 12 },
  { nom: 'Souris', prix: 24.5, stock: 0 },
  { nom: 'Écran', prix: 279, stock: 4 },
  { nom: 'Webcam', prix: 59.99, stock: 7 },
  { nom: 'Casque', prix: 89, stock: 0 },
  { nom: 'Hub USB', prix: 34.9, stock: 15 }
];
```

Exercices - Récapitulatif (2/4)

- Écrivez une fonction « afficherProduits(liste) » qui vide le <ul>, puis crée un <li> par produit de la liste reçue, au format « Clavier - 49.90 € - 12 en stock ».
- Si le stock d'un produit vaut 0, affichez « rupture de stock » à la place de la quantité et ajoutez la classe CSS « rupture » au <li>.
- Écrivez une fonction « valeurStock(liste) » qui retourne la somme des prix × stock des produits reçus.
- À la fin de « afficherProduits », affichez dans le paragraphe « resume » le nombre de produits et la valeur du stock : « 6 produits - valeur du stock : 2658.23 € ».
- Appelez « afficherProduits(produits) » au chargement de la page.

Exercices - Récapitulatif (3/4)

- Ajoutez les boutons « Tous », « En stock », « Prix croissant », « Prix décroissant » et « Nom A→Z », qui appellent « afficherProduits » avec la liste filtrée ou triée.
- Triez toujours une copie du tableau (« slice ») : après un tri, le bouton « Tous » doit réafficher l'ordre d'origine.
- Ajoutez un champ texte et un bouton « Rechercher » qui n'affiche que les produits dont le nom contient le texte saisi, sans tenir compte des majuscules.
- Ajoutez un bouton « Supprimer » dans chaque <li> : il retire le produit du tableau « produits » et réaffiche tout le catalogue.
- Écrivez tous vos callbacks en fonctions fléchées.

Exercices - Récapitulatif (4/4)

- Vérifiez vos résultats :
  - « En stock » → 4 produits, valeur du stock : 2658.23 €
  - « Prix croissant » → Souris en premier, Écran en dernier
  - « Nom A→Z » → Écran entre Clavier et Hub USB
  - recherche « CA » → Webcam et Casque
  - suppression de l'Écran → « 5 produits - valeur du stock : 1542.23 € »
- Expliquez pourquoi la valeur du stock ne change pas quand on n'affiche que les produits en stock.
- Pour aller plus loin : ajoutez trois champs (nom, prix, stock) et un bouton « Ajouter » qui ajoute un produit au catalogue.

Les timers

Timeout

Les timers

Déclencher un timer « Timeout »

La méthode « setTimeout » permet l'appel d'une fonction après un délai.

```
const timeoutId = setTimeout(functionRef, delay, param1, param2, /* …, */ paramN);
```

Cette méthode prend en arguments :

- functionRef (obligatoire)
  La fonction à exécuter.
- delay
  Le délai à appliquer.
- param1, ..., paramN
  Les arguments à envoyer à la fonction exécutée.

Cette méthode retourne un entier positif qui représente le « Timeout Id »

Arrêter un timer « Timeout »

La méthode « clearTimeout » permet d'annuler un timer généré par « setTimeout ».

```
clearTimeout(timeoutId);
```

Cette méthode prend en argument le « Timeout Id » du timer à annuler.

Remarque : Si le paramètre « Timeout Id » fourni n'identifie pas un timer précédemment créé, l'appel de cette méthode ne fait rien.

Interval

Les timers

Déclencher un timer « Interval »

La méthode « setInterval » permet l'appel répété d'une fonction espacé d'un délai fixe.

```
const intervalId = setInterval(functionRef, delay, param1, param2, /* …, */ paramN);
```

Cette méthode prend en arguments :

- functionRef (obligatoire)
  La fonction à exécuter.
- delay
  Le délai à appliquer.
- param1, ..., paramN
  Les arguments à envoyer à la fonction exécutée.

Cette méthode retourne un entier positif qui représente le « Interval Id »

Arrêter un timer « Interval »

La méthode « clearInterval » permet d'annuler un timer généré par « setInterval ».

```
clearInterval(intervalId);
```

Cette méthode prend en argument le « Interval Id » du timer à annuler.

Remarque : Si le paramètre « Interval Id » fourni n'identifie pas un timer précédemment créé, l'appel de cette méthode ne fait rien.

Exercices

Les timers

Exercices - Manipulation de timers

Créez une page HTML avec une horloge numérique simple :

- Affichez l'heure courante (heures, minutes, secondes) dans un <p id="horloge">.
- Utilisez setInterval pour mettre l'affichage à jour chaque seconde.
- Formatez l'affichage sous la forme HH:MM:SS (avec un zéro devant si nécessaire, ex : 08:05:03).
- Ajoutez un bouton "Pause" qui suspend l'horloge avec clearInterval, et un bouton "Reprendre" qui la relance.

Interaction avec le DOM
- Les événements -

Abonnement aux events

Interaction avec le DOM - Les événements

Interagir aux événements d'une page Web

Les éléments d'une page Web peuvent provoquer des événements.

L'API de DOM permet au JavaScript de réagir à ceux-ci à l'aide de fonction.

Pour s'abonner à un event, il est possible d'utiliser deux techniques :

- La propriété de gestion d'event (Spécification du DOM 0 - Ancienne manière)
  Permet de remplacer l'écouteur d'événement existant de l'élément.
- Utilisation des Event Listeners (Spécification du DOM 2)
  Permet d'ajouter ou retirer l'enregistrement d'un nouvel écouteur d'événements.

La propriété de gestion d'évent - Via le code HTML

Pour lier une fonction à un event depuis le code HTML, il faut :

- Créer une fonction dans le code JS
- Définir un attribut commençant par « on… » suivit du type d'event dans la balise.
- Affecter à cet attribut, le nom de la fonction sans oublier les parenthèses.

• HTML

```html
<button onclick="handleDemoEvent()">Click Here !</button>
```

• JavaScript

```javascript
function handleDemoEvent() {
  // Traitement ...
}
```

La propriété de gestion d'évent - Via le code JS

Pour lier une fonction à un event à l'aide de la propriété de gestion d'event, il faut :

- Récupérer la balise HTML dans le code JS.
- Utiliser une propriété d'event commençant par « on... » suivi du type d'event.
- Affecter à cette propriété, le callback à exécuter.

```
const btnDemo = document.getElementById('btn-demo');

btnDemo.onclick = handleDemoEvent;
// ou
btnDemo.onclick = function () {
  // Traitement ...
}
```

Les Event Listeners

S'abonner à des events à l'aide de « Event Listener » est la manière d'enregistrer un écouteur d'événements telle que spécifiée dans le DOM du W3C.

Les Event Listeners ont comme avantages :

- de pouvoir ajouter plusieurs « listener » sur un même événement,
- de supprimer les instances dupliquées si des « listener » identiques sont ajoutés plusieurs fois avec les mêmes paramètres,
- de fonctionner avec tout élément DOM (pas uniquement les éléments HTML).
- de pouvoir retirer un ou plusieurs « listener » d'un élément.

Les Event Listeners - S'abonner à un événement

La méthode « addEventListener » permet d'ajouter un « Listener » à un élément.

Cette méthode prend en arguments :

- Le type d'event
- Une fonction de callback
- Un objet « options » ou un booléen « useCapture » (Optionnel)

```
const btnDemo = document.getElementById('btn-demo');

btnDemo.addEventListener('click', handleDemoEvent);
// ou
btnDemo.addEventListener('click', function () {
  // Traitement ...
});
```

Les Event Listeners - S'abonner à un événement

L'objet « options » permet de spécifier les caractéristiques du « listener».

Les options disponibles sont :

- ➔ once (Booléen)
  Permet d'indiquer que le listener doit être supprimé après son premier appel
- ➔ passive (Booléen)
  Permet de spécifier que le listener ne doit pas appeler la méthode « preventDefault »
- ➔ capture (Booléen)
  Permet de définir que l'événement sera distribué au « Listener » avant d'être distribué aux « EventTarget » situés en dessous dans l'arborescence DOM.

Les Event Listeners - Se désabonner à un événement

La méthode « removeEventListener » permet de retirer un « Listener » à un élément.

Cette méthode prend en arguments :

- Le type d'event
- Une fonction de callback
- Un objet « options » ou un booléen « useCapture » (Optionnel)

```
const btnEvent = document.getElementById('btn-event');

btnEvent.removeEventListener('click', handleDemoEvent);
```

Remarque : Si l'abonnement du listener a été configuré avec l'option de « capture » *(via l'objet options ou useCapture)*, il sera nécessaire de le définir lors du désabonnement.

Variable de l'événement

Interaction avec le DOM - Les événements

La variable de l'événement

Une fonction abonnée à un événement reçoit en paramètre la variable d'évent.

Cette variable permet :

- D'accéder à des information relative à l'event
- Déclencher des comportement sur l'évent.

```
const btnEvent = document.getElementById('btn-event');

btnEvent.addEventListener('click', (event) => {
  console.log('Variable d\'event', event);

});
```

Quelques propriétés et méthode de l'événement

- .type : Obtenir le type d'événement déclenché.
- .target : Identifie l'élément à partir duquel l'événement s'est déclenché.
- .currentTarget: Identifié la cible pour l'événement (via la propagation). Fait référence à l'élément sur lequel l'abonnement a été réalisé.

- .preventDefault() : Annule l'action par défaut de l'événement.
- .stopPropagation() : Arrête la propagation de l'événement aux éléments parents.
- .stopImmediatePropagation() : Arrête la propagation de l'événement à élément en cours et aux éléments parents.

Types d’events des éléments du DOM

Interaction avec le DOM - Les événements

Liste non exhaustive des events

- click

Clique sur un élément.

- dblclick

Double clique sur un élément.

- contextmenu

Ouverture du menu via le clique droit.

- keydown

Une touche est pressée.

- keyup

Une touche est relâchée.

- ~~keypress~~ (déprécié)

Une touche de caractère est pressée.

Liste non exhaustive des events

- focus / focusin

L'élément reçoit le focus.

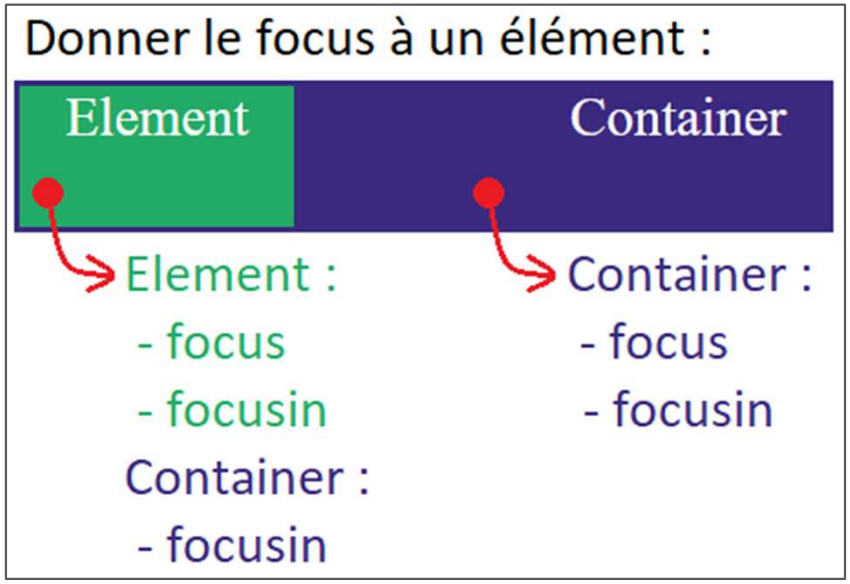
- blur / focusout

L'élément perd le focus.

- mousemove

Le curseur est déplacé.

La différence entre les évents de focus est que « focus / blur » ne se propage pas, à l'inverse de « focusin / focusout ».



Liste non exhaustive des events

- change

La valeur d'une balise a été modifiée.

- input

La valeur d'une balise est modifié,
cet event se déclenche en temps réel.

- invalid

Le contenu d'un balise input est invalide.

- submit

Le formulaire a été validé.

- reset

Le formulaire a été réinitialisé.

- formdata

Génération d'un objet « FormData », celui-ci contient les données du form.

Types d’events de « Window »

Interaction avec le DOM - Les événements

Liste non exhaustive des events de Window

- load

La page et toutes ses dépendances sont complètement chargées.

- beforeunload

La page va être déchargée. Permet d'interrompre la fermeture de celle-ci.

- languagechange

La langue préférée (via le navigateur) de l'utilisateur a été changée.

- online

Le navigateur a obtenu l'accès au réseau et « Navigator.onLine » passe à true.

- offline

Le navigateur perd l'accès au réseau et « Navigator.onLine » bascule à false.

Exercices

Interaction avec le DOM - Les événements

Exercices - Jeu du Memory (1/2)

Mise en place

- Créez un tableau de 8 paires de valeurs (ex : des émojis, des lettres ou des images) et mélangez-les aléatoirement pour obtenir 16 cartes.
- Générez dynamiquement 16 cases dans une grille.
  Chaque case affiche "?" par défaut (carte face cachée).

Comportement au clic

- Au clic sur une carte, révélez sa valeur.
- Empêchez de cliquer sur une carte déjà retournée ou déjà trouvée.

Exercices - Jeu du Memory (2/2)

Logique de comparaison

- Lorsque deux cartes sont retournées, comparez-les :
  - Si elles sont identiques : elles restent visibles (paire trouvée).
  - Si elles sont différentes : après un court délai (setTimeout), retournez-les à nouveau face cachée.

Fin de partie

- Lorsque toutes les paires sont trouvées, affichez un message de victoire dans la page avec le nombre de tentatives effectuées.

Bonus : Ajoutez un bouton "Rejouer" qui réinitialise et remélange les cartes.

Les exceptions

L’instruction « try ... catch ... finally »

Les exceptions

L'instruction « try ... catch ... finally »

Le « try » permet d'exécuter un bloc de code dans lequel une erreur pourrait survenir.

Cette instruction ne peut pas être utilisée seule. Celle-ci doit être suivie de minimum une des clauses suivantes :

- La clause « catch » : Code permettant de réagir à l'exception.
- La clause « finally » : Code qui sera toujours exécuté.

Il peut donc y avoir trois formes suivantes pour cette instruction :

➔ try ... catch
➔ try ... finally
➔ try ... catch ... finally

La clause « catch »

La clause « catch » permet de définir un bloc de code qui s'exécutera si une exception se déclenche dans le « try ».

```
try {
  myProcess(); // Fonction pouvant générer une exception
}
catch (err) {
  console.error('Une erreur a été provoquée par « myProcess »');
  console.error(err);
}
```

La clause « catch »

S'il est nécessaire de réaliser un traitement différent en fonction du type d'erreur, il faut utiliser une structure conditionnelle avec l'opérateur « instanceOf ».

```
try {
  myProcess(); // Fonction pouvant générer plusieurs types d'exceptions
}
catch (e) {
  if (e instanceof TypeError) {
    // Gestion de l'exception TypeError
  }
  else if (e instanceof RangeError) {
    // Gestion de l'exception RangeError
  }
  else {
    // Gestion des exceptions non spécifiées
  }
}
```

La clause « finally »

La clause « finally » permet de définir du code qui s'exécutera après l'exécution du code du « try » et de la clause « catch », si celle-ci est présente.

```
let fileStream = null;
try {
  // Accède à une ressource
  fileStream = readFile('...');
  // Utilisation de la ressource
  fileStream.write(data);
}
finally {
  // Toujours libérer la ressource
  fileStream?.close();
}
```

Déclencher une exception

Les exceptions

Déclencher une exception

L'instruction « throw » permet de déclencher une exception.

Celle-ci entraîne l'arrêt de l'exécution de la fonction actuelle et le premier bloc « catch » rencontré reprend la suite de l'exécution du script.
Si aucun « catch » n'a été trouvé, l'exécution du script se termine.

Lorsqu'on souhaite déclencher une exception, il est nécessaire de définir une expression avec la valeur à renvoyer

```
throw new Error("Boum 🧨");
throw "Unexpected error";
throw 42;
```

Déclencher une exception - Best Practice

Bien qu'il n'y ai pas de limitation de type pour l'erreur envoyée par le « throw », la valeur envoyée par l'exception devrait toujours être d'un type « Error ».

La raison étant que le code qui traite l'exception pourrait s'attendre à utiliser certaines propriétés de l'objet « Error ».

Les types d'exceptions pour du JS client :

- Les ECMAScript exceptions.
- La « DOMException »

Liste des exceptions de ECMAScript :

- Error
- EvalError
- RangeError
- ReferenceError
- SyntaxError
- TypeError
- URIError
- AggregateError

Exercices

Les exceptions

Exercices - Les exceptions

- Écrivez une fonction « diviser(a, b) » qui lève une erreur si b vaut 0.
- Appelez cette fonction dans un try...catch et affichez le message d'erreur. Ajoutez un finally qui affiche 'Opération terminée' dans tous les cas.
- Créez une fonction « convertirEnNombre(valeur) » qui lève une erreur si :
  - la valeur convertie est NaN → TypeError
  - la valeur est supérieure aux entiers « safe » du JS → RangeError
- Bonus : Utilisez la fonction « convertirEnNombre » dans un seul try...catch et traitez différemment un TypeError et un RangeError.

Interaction avec le DOM
- Les formulaires -

Objectif des formulaires

Interaction avec le DOM - Les formulaires

Objectif des formulaires

Les formulaires sont utilisés pour qu'un utilisateur puisse interagir avec une page web.

Ils permettent d'envoyer des données au serveur.

➔ Par défaut, via le comportement de la balise « form » (en « GET » ou « POST »)

➔ En JS, via des requêtes Ajax (abordé ultérieurement)

Le langage JavaScript permet :

- D'accéder aux données du formulaire (en lecture et écriture)
- D'interrompre ou de déclencher les comportements « Submit » et « Reset »
- D'ajouter des règles de validation sur les différentes inputs

Structure d'un formulaire

Les formulaires sont généralement constitués :

- D'une balise « form »
- De balise de saisie pour l'utilisateur
  - input (text, number, radio, checkbox, ...)
  - textarea
  - select
- D'un bouton de validation et d'annulation

Structure du formulaire utilisé dans le support

```html
<form id="example-form" name="registration" method="post">
  <div>
    <label for="field-email">Email</label>
    <input id="field-email" type="email" name="email" required>
  </div>
  <div>
    <label for="field-pseudo">Pseudo</label>
    <input id="field-pseudo" type="text" name="pseudo">
  </div>
  <div>
    <label for="field-guest">Nombre de personne</label>
    <input id="field-guest" type="number" name="nbGuest" min="1" required>
  </div>
  <div>
    <button type="submit">Valider l'inscription</button>
    <button type="reset">Réinitialiser</button>
  </div>
</form>
```

Manipulation les formulaires

Interaction avec le DOM - Les formulaires

Récupérer un formulaire

Pour récupérer une balise de formulaire, il existe de plusieurs possibilité :

- À l'aide des méthodes « getElementById » et « querySelector ».
- Via la propriété « forms » de l'objet « document » (via son nom ou son index).
- En utilisant le nom du formulaire en tant que propriété de « document ».

```
const form1 = document.getElementById('example-form');
const form2 = document.querySelector('#example-form');

const form3 = document.forms['registration'];
const form4 = document.forms[0];

const form5 = document.registration;
```

Récupérer les champs d'un formulaire

La propriété « elements » d'un objet de type « HTMLFormElement » permet d'obtenir une collection qui contient tous les champs (input, textarea, button) du formulaire.

- Javascript

```
for (const elem of form1.elements) {
  console.log(elem);
}
```

- Résultat

```
<input id="field-email" type="email" name="email" required>
<input id="field-pseudo" type="text" name="pseudo">
<input id="field-guest" type="number" name="nbGuest" min="1" required>
<button type="submit">Valider l'inscription</button>
<button type="reset">Réinitialiser</button>
```

Récupérer un champ d'un formulaire

Pour récupérer un des champs d'un formulaire, il est possible de l'obtenir :

- Depuis la propriété « element ».
- Directement depuis l'objet formulaire.

Ces deux méthodes fonctionnent avec le nom et l'index du champ ciblé.

```
const fieldPseudo1 = form1.elements['pseudo'];
const fieldPseudo2 = form1.elements[1];

const fieldPseudo3 = form1['pseudo'];
const fieldPseudo4 = form1[1];
```

Interagir avec les évents d'un formulaire

L'event « submit » permet d'interagir sur la validation d'un formulaire avant que les données de celui-ci ne soit envoyé.

```
form1.addEventListener('submit', (event) => {
  // Traitement ...
});
```

L'event « reset » permet d'interagir sur la remise à zéro d'un formulaire avant que les données ne soit effacé.

```
form1.addEventListener('reset', (event) => {
  // Traitement ...
});
```

Rappel : L'utilisation de « event.preventDefault() » annule leurs comportements.

Interagir avec les évents d'un formulaire

L'event « formdata » se déclenche quand l'objet avec les valeurs du formulaire est créé.
Il est donc possible d'ajouter, de modifier et de supprimer des valeurs du formData.

```
form1.addEventListener('formdata', (event) => {
  event.formData.append('message', 'The cake is a lie 🍰');
});
```

Données reçues par le serveur

```
{
  email: 'della.duck@demo.com',
  pseudo: 'Della',
  nbGuest: '4',
  message: 'The cake is a lie 🍰'
}
```

Attention, cette événement n'est pas annulable !

Exercices

Interaction avec le DOM - Les formulaires

Exercices - Manipulation de formulaires - HTML (1/2)

Créez un formulaire HTML contenant :

- Un <input type="text"> pour le prénom
- Un <input type="number"> pour l'âge
- Un <select> pour choisir un pays (au moins 3 options)
- Une <input type="checkbox"> pour accepter les conditions d'utilisation
- Un bouton de soumission

Exercices - Manipulation de formulaires - JS (2/2)

Sur le formulaire créer, ajouter du javascript pour :

- Interceptez l'event submit et empêchez le comportement par défaut.
- Validez que le prénom contient au moins 2 caractères.
  Affichez un message d'erreur sous le champ si ce n'est pas le cas.
- Validez que l'âge est compris entre 18 et 99.
  Affichez un message d'erreur sous le champ si ce n'est pas le cas.
- Vérifiez que la checkbox est cochée.
  Affichez un message d'erreur si ce n'est pas le cas.
- Si tout est valide, affichez un récapitulatif des données saisies dans un sous le formulaire.

Les promesses

Déclaration et utilisation

Les promesses

C’est quoi une promesse ?

Le mécanisme des promesses permet à une méthode de renvoyer un objet de type « Promise », alors que celle-ci n’a pas encore terminé sa tâche.

Cet objet représente littéralement la promesse faite par cette méthode de transmettre un résultat une fois sa tâche accomplie.

Les promesses sont généralement utilisées pour réaliser des actions asynchrones en JavaScript et cela permet d’éviter d’avoir recours au mécanisme des callbacks.

L’objet de type « Promise »

Un objet « promise » représente donc une valeur qui peut être disponible maintenant, dans le futur, voire jamais.

Cet objet peut être dans l'un de ces trois états :

- pending (en attente) : la promesse est en cours de traitement.
- fulfilled (tenue) : la promesse est résolue et a réussi.
- rejected (rompue) : la promesse est résolue et a échoué.

Création d'une promesse

Pour créer un objet du type « Promise », il est nécessaire de l'instancier à l'aide de son constructeur en lui fournissant en paramètre une fonction de callback.

Cette fonction contiendra le code à exécuter et possède jusqu'à deux paramètres :

- resolve :

  Promesse « fulfilled »

  Fonction pour terminer une promesse avec succès.
  Celle-ci prend un paramètre pour renvoyer une valeur.

- reject :

  Promesse « rejected »

  Fonction pour terminer une promesse avec une erreur.
  Celle-ci prend un paramètre pour renvoyer la raison de l'échec de la promesse.

Exemple de création d'une fonction avec une promesse

```
function examplePromise(dividend, divisor) {
  // La fonction renvoie un objet du type « Promise »
  return new Promise((resolve, reject) => {
    // Utilisation d'un timer de 500 ms pour "simuler" du code asynchrone
    setTimeout(() => {
      // Promesse rompue si le diviseur vaut zéro
      if(divisor === 0) {
        reject(new Error('Division by zero !'))
      }
      // Traitement
      const result = dividend / divisor;
      // Promesse tenue
      resolve(result);
    }, 500);
  });
}
```

Utilisation d'une promesse

Pour utiliser une promesse, il est possible d'utiliser trois méthodes sur cet objet :

- .then(resolveCallback [, rejectCallback])
  Permet de traiter le succès et l'erreur (si le callback est défini) d'une promesse.
- .catch(rejectCallback)
  Permet de traiter l'erreur d'une promesse.
- .finally(finallyCallback)
  Permet de traiter la complétion d'une promesse (en succès ou en erreur).

Ces méthodes renvoient également une promesse, ce qui permet de les enchaîner.

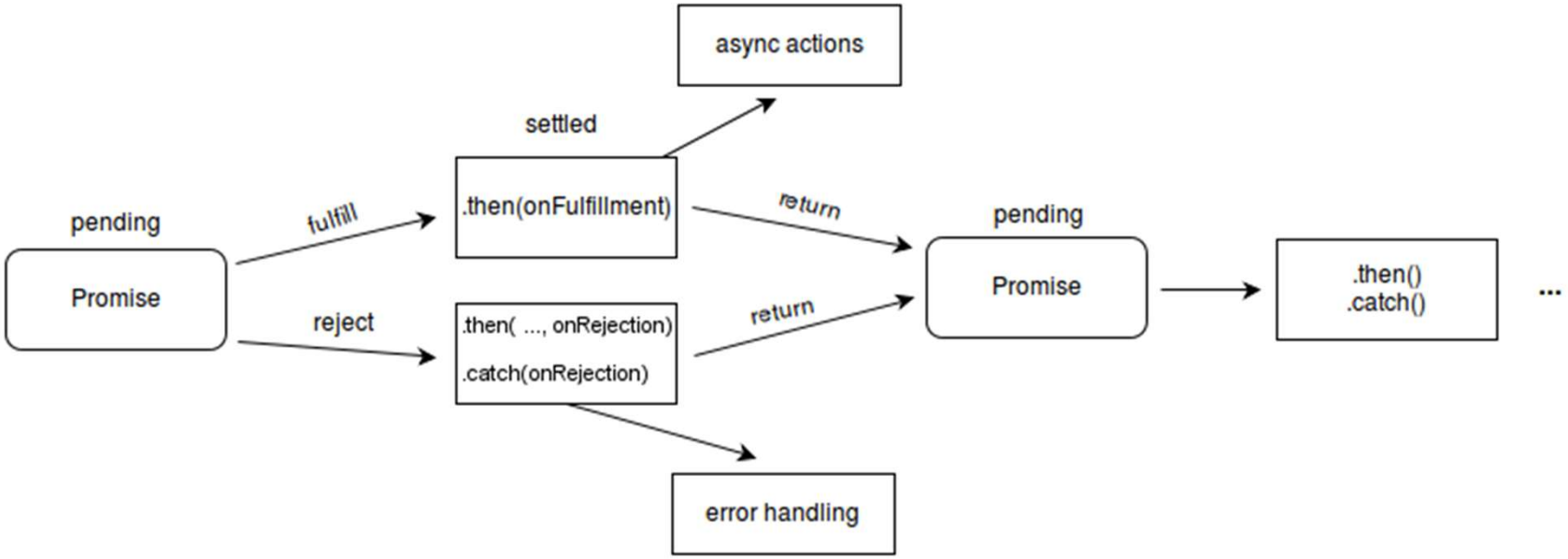
Exemple d'utilisation d'une promesse

```
const dividend = inputDividend.valueAsNumber;
const divisor = inputDivisor.valueAsNumber;

examplePromise(dividend, divisor)
  .then((data) => {
    result.value = `Le resultat de ${dividend} / ${divisor} est ${data}`;
  })
  .catch((reason) => {
    result.value = `Erreur : ${reason.message}`;
  })
  .finally(() => {
    console.log('Traitement de la promesse terminé !')
  });
```

Enchaîner des promesses

Les méthodes « then», « catch » et « finally » renvoient toutes des promesses.
Ce qui permet de pouvoir les chaînées.



Enchaîner des promesses

Exemple avec quatre promesses chaînés

```
function exampleChainPromise(nb) {
  return addition(nb, 3)
    .then(r1 => multi2(r1))
    .then(r2 => addition(r2, 42))
    .then(r3 => multi3(r3))
    .then(rf => {
      return `Resultat : ${rf}`;
    })
    .catch(err => {
      return 'Erreur...';
    });
}
```

```
function addition(nb1, nb2) {
  return new Promise((resolve) => {
    const result = Math.round(nb1 + nb2);
    setTimeout(resolve, 500, result);
  })
}

function multi2 (nb) {
  return new Promise((resolve) => {
    const result = nb * 3;
    setTimeout(resolve, 500, result);
  })
}

function multi3 (nb) {
  return new Promise((resolve) => {
    const result = nb * 3;
    setTimeout(resolve, 1_000, result);
  })
}
```

Les méthodes des promesses

Les promesses

Méthode pour créer des promesses terminées

- Promise.resolve([data])

Créer un objet du type « Promise » qui est résolu avec la valeur donnée.

- Promise.reject([reason])

Créer un objet du type « Promise » qui est rejeté avec la cause.

Méthode pour utiliser des promesses en parallèle

- Promise.all(promiseCollection)

Créer une promesse qui attend que toutes les promesses reçues soit résolue.

Elle renvoie le résultat de chaque promesse dans une collection.
Dès qu'une des promesse est rejeté, celle-ci passe en état « rejected ».

Si la collection reçu est vide, la promesse sera résolue avec une collection vide.

```
const p1 = new Promise((resolve, reject) => { setTimeout(() => { resolve('Promesse 1'); }, 200); });
const p2 = new Promise((resolve, reject) => { setTimeout(() => { resolve('Promesse 2'); }, 50); });
const p3 = new Promise((resolve, reject) => { setTimeout(() => { reject(new Error('Boum !')); }, 100); });

Promise.all([p1, p2]).then(data => console.log(data)).catch(error => console.log(error));
// Resultat → ['Promesse 1', 'Promesse 2']

Promise.all([p1, p3]).then(data => console.log(data)).catch(error => console.log(error));
// Resultat → {Error: 'Boum !'}
```

Méthode pour utiliser des promesses en parallèle

- Promise.allSettled(promiseCollection)

Créer une promesse qui attend que toutes les promesses reçues soit terminé.

Elle renvoie le résultat de chaque promesse (fulfilled ou rejected) dans une collection.

Si la collection reçu est vide, la promesse sera résolue avec une collection vide.

```
const p1 = new Promise((resolve, reject) => { setTimeout(() => { resolve('Promesse 1'); }, 200); });
const p2 = new Promise((resolve, reject) => { setTimeout(() => { resolve('Promesse 2'); }, 50); });
const p3 = new Promise((resolve, reject) => { setTimeout(() => { reject(new Error('Boum !')); }, 100); });

Promise.allSettled([p1, p2]).then(data => console.log(data)).catch(error => console.log(error));
// Resultat → [{status: 'fulfilled', value: 'Promesse 1'}, {status: 'fulfilled', value: 'Promesse 2'}]

Promise.allSettled([p1, p3]).then(data => console.log(data)).catch(error => console.log(error));
// Resultat → [{status: 'fulfilled', value: 'Promesse 1'}, {status: 'rejected', reason: {Error: 'Boum !'}}]
```

Méthode pour utiliser des promesses en parallèle

- Promise.any(promiseCollection)

Créer une promesse qui attend qu'une des promesses reçues soit résolue.

Elle renvoie le résultat de la première promesse résolue.
Si toutes les promesses sont rejetées, celle-ci passe en état « rejected ».

Si la collection reçu est vide, la promesse sera rejeté avec une "AggregateError".

```
const p1 = new Promise((resolve, reject) => { setTimeout(() => { resolve('Promesse 1'); }, 200); });
const p2 = new Promise((resolve, reject) => { setTimeout(() => { resolve('Promesse 2'); }, 50); });
const p3 = new Promise((resolve, reject) => { setTimeout(() => { reject(new Error('Boum !')); }, 100); });

Promise.any([p1, p2]).then(data => console.log(data)).catch(error => console.log(error));
// Resultat → 'Promesse 2'

Promise.any([p1, p3]).then(data => console.log(data)).catch(error => console.log(error));
// Resultat → 'Promesse 1'
```

Méthode pour utiliser des promesses en parallèle

- Promise.race(promiseCollection)

Créer une promesse qui attend qu'une des promesses reçues soit terminée.

Elle renvoie le résultat de la première promesse terminé (fulfilled ou rejected).

Si la collection reçu est vide, la promesse sera continuellement en attente.

```
const p1 = new Promise((resolve, reject) => { setTimeout(() => { resolve('Promesse 1'); }, 200); });
const p2 = new Promise((resolve, reject) => { setTimeout(() => { resolve('Promesse 2'); }, 50); });
const p3 = new Promise((resolve, reject) => { setTimeout(() => { reject(new Error('Boum !')); }, 100); });

Promise.race([p1, p2]).then(data => console.log(data)).catch(error => console.log(error));
// Resultat → 'Promesse 2'

Promise.race([p1, p3]).then(data => console.log(data)).catch(error => console.log(error));
// Resultat → {Error: 'Boum !'}
```

Les fonctions asynchrones

Les promesses

Les fonctions asynchrones

Une fonction peut être déclaré en tant que fonction asynchrone, pour cela, il est nécessaire de la précéder de « async ».

Celle-ci peut contenir des promesses précédées du mot clef « await ».
Ce qui interrompt son exécution et attend la résolution de la promesse.

La fonction renvoie une promesse :

- Résolue lorsque elle est terminé.
- Rompue en cas d'exception.

```
// Fonction qui renvoie une promesse
function doSomething() {
  return new Promise((resolve) => {
    setTimeout(resolve, 2_000, '42')
  });
}

// Fonction Asynchrone
async function exampleAsyncAwait() {
  console.log('Début du traitement...');

  // Traitement durant de 2s
  const result = await doSomething();

  console.log(`Résutlat : ${result}`);
}
```

Les fonctions asynchrones

Les fonctions asynchrone permettent de simplifier l'utilisation des promesses.
Elle évite de devoir configurer explicitement les chaînes de promesse.

Pour traiter les exceptions provoqués par des promesses en attente (via await) lorsque celles-ci sont rompu, il est nécessaire d'utiliser un bloc « try / catch ».

```
async function exampleErrorHandling() {
  let result;
  try {
    result = await doSomething();
  }
  catch(err) {
    result = 'Erreur lors du traitement';
  }
  return result;
}
```

Enchaîner des promesses avec async/await

Exemple avec quatre promesses chaînés

```
async function exampleSequentialPromise(nb) {
  try {
    const r1 = await addition(nb, 3);
    const r2 = await multi2(r1);
    const r3 = await addition(r2, 42);
    const rf = await multi2(r3);
    console.log(`Resultat : ${rf}`);
  }
  catch (err) {
    console.log('Erreur...');
  }
}
```

```
function addition(nb1, nb2) {
  return new Promise((resolve) => {
    const result = Math.round(nb1 + nb2);
    setTimeout(resolve, 500, result);
  })
}

function multi2 (nb) {
  return new Promise((resolve) => {
    const result = nb * 3;
    setTimeout(resolve, 500, result);
  })
}

function multi3 (nb) {
  return new Promise((resolve) => {
    const result = nb * 3;
    setTimeout(resolve, 1_000, result);
  })
}
```

Exercices

Les promesses

Exercices - Les promesses

Dans ce combat au tour par tour, chaque action du joueur est simulée par une promesse.

- Créez une fonction concentration(ms) qui retourne une promesse et simule le temps de préparation du joueur.
  Utilisez-la pour afficher le message « Le joueur est prêt ! » une fois le délai écoulé.
- Créez une fonction attaque() qui retourne une promesse dont le résultat est incertain : elle peut réussir ou échouer aléatoirement. Un message doit être affiché selon l'issue.
- Créez une fonction async jouerTour() qui orchestre un tour complet : le joueur se concentre pendant un temps aléatoire entre 2 et 5 secondes, puis lance son attaque.
- Bonus : Simulez un combat où quatre joueurs agissent en même temps. Lancez leurs attaques en parallèle et affichez un résumé des résultats une fois que tous ont terminé.

Les requêtes HTTP

La fonction « fetch »

Les requêtes HTTP

Interroger une API avec « fetch »

« fetch » envoie une requête HTTP à une adresse (URL) et renvoie aussitôt une promesse : la réponse, souvent du JSON, arrivera plus tard.

```
const API = 'https://pokeapi.co/api/v2';

const promesse = fetch(`${API}/pokemon/pikachu`);
console.log(promesse);
// Promise {<pending>}
```

Lire la réponse avec « await »

Dans une fonction « async », « await » attend la fin d'une promesse et renvoie sa valeur.
La méthode « json » de la réponse est elle aussi asynchrone.

```
const API = 'https://pokeapi.co/api/v2';

async function chargerPokemon(nom) {
  const reponse = await fetch(`${API}/pokemon/${nom}`);
  const pokemon = await reponse.json();
  console.log(pokemon.name, pokemon.height);
}

chargerPokemon('pikachu');
// pikachu 4
```

Vérifier le statut de la réponse

Une réponse 404 ou 500 n'est pas une erreur pour « fetch ». Vérifiez « reponse.ok » (vrai pour un statut de 200 à 299) avant de lire les données.

```
async function chargerPokemon(nom) {
  const reponse = await fetch(`${API}/pokemon/${nom}`);
  if (!reponse.ok) {
    console.log(`Erreur ${reponse.status}`);
    return;
  }
  const pokemon = await reponse.json();
  console.log(pokemon.name);
}

chargerPokemon('pikachuu');
// Erreur 404
```

Gérer les erreurs réseau

« fetch » échoue seulement si le serveur est injoignable (pas de réseau, adresse inconnue). Un « try … catch » permet alors d'afficher un message.

```
async function chargerPokemon(nom) {
  try {
    const reponse = await fetch(`${API}/pokemon/${nom}`);
    if (!reponse.ok) {
      console.log(`Erreur ${reponse.status}`);
      return;
    }
    const pokemon = await reponse.json();
    console.log(pokemon.name);
  } catch (erreur) {
    console.log('Impossible de joindre l'API.');
  }
}
chargerPokemon('pikachu');
// Sans réseau : Impossible de joindre l'API.
```

L'ordre d'exécution

Le code qui suit l'appel d'une fonction asynchrone s'exécute sans attendre la réponse : affichez un message « Chargement... » pendant la requête.

```
async function chargerPokemon(nom) {
  const reponse = await fetch(`${API}/pokemon/${nom}`);
  const pokemon = await reponse.json();
  console.log(pokemon.name, pokemon.height);
}

console.log('avant');
chargerPokemon('pikachu');
console.log('après');
// avant
// après
// pikachu 4
```

À retenir

- « fetch » renvoie une promesse : lisez la réponse avec « await », dans une fonction « async ».
- Une réponse 404 n'est pas une erreur pour « fetch » : vérifiez toujours « reponse.ok ».
- « try ... catch » intercepte les erreurs réseau : prévoyez un message pour l'utilisateur.

Annexes

La collection à clefs « Map »

Annexes

Les objets de type « Map »

Un objet « Map » permet de stocker des éléments sous la forme de « Clef / Valeur ».
Les clefs peuvent être de n'importe quel type de valeur (primitive ou objet).

Celui-ci est un itérable, il peut être parcouru à l'aide d'une boucle.
L'ordre d'insertion des éléments est mémorisé.

Aperçu d'un objet « Map » :

```
const dico = new Map();

dico.set('Pomme', 250);
dico.set('Poire', 300);
dico.set('Mangues', 70);
```

```
▶ Map(3) {'Pomme' => 250, 'Poire' => 300, 'Mangues' => 70}
```

Méthodes sur un objet de type « Map »

- .set(clef, valeur)

Permet d'ajouter une combinaison clef / valeur dans l'objet « Map ».
Si la clef existe déjà, la valeur sera mise à jour avec celle reçue.

- .get(clef)

Renvoie la valeur liée à la clef. Si aucune valeur n'est trouvée, renvoie « undefined ».

- .has(clef)

Renvoie un booléen pour indiquer si la clef est présente dans l'objet « Map ».

Méthodes sur un objet de type « Map »

- .delete(clef)

Permet de supprimer un élément à l'aide de sa clef.
Cette méthode renvoie « true » si l'élément existe et qu'il a bien été supprimé.

- .clear()

Permet de supprimer tous les éléments.

- .forEach(méthode)

Permet d'effectuer un traitement itératif. *(Identique à la méthode forEach() des Array)*

Propriété sur un objet de type « Map »

La propriété « size » permet d'obtenir le nombre d'éléments de l'objet « Map ».

```
const dico = new Map();
const s1 = dico.size;   // → 0

dico.set('z1', 'Zaza');
dico.set('d1', 'Donald');
dico.set('r1', 'Riri');
const s2 = dico.size;   // → 3

dico.set('d1', 'Della');
dico.delete('r1');
const s3 = dico.size;   // → 2

dico.clear();
const s4 = dico.size;   // → 0
```

Exemple de l'utilisation d'un objet « Map »

```
// Déclaration
const students = new Map();

// Ajouter des éléments (clef : string / valeur : objet)
students.set('zava42', { firsname: 'Zaza', lastname: 'Vanderquack' });
students.set('ridu19', { firsname: 'Riri', lastname: 'Duck' });

// Obtenir le nombre d'élément
const nbElem = students.size; // → 2

// Obtenir des valeurs via leur clef
const v1 = students.get('zava42'); // → L'objet : {firsname: 'Zaza', lastname: 'Vanderquack'}
const v2 = students.get('cama01'); // → undefined

// Parcourir les éléments
for(const [key, value] of students) {
  console.log(`${key} => ${value}`);
}
```

La collection à clefs « Set »

Annexes

Les objets de type « Set »

Un objet « Set » permet de stocker des éléments sous la forme de clef unique.
Les clefs peuvent être de n'importe quel type de valeur (primitive ou objet).

Celui-ci est un itérable, il peut être parcouru à l'aide d'une boucle.
L'ordre d'ajout des éléments est mémorisé.

Aperçu d'un objet « Set » :

```
const fruits = new Set();

fruits.add('Pomme');
fruits.add('Poire');
fruits.add('Mangues');

▶ Set(3) {'Pomme', 'Poire', 'Mangues'}
```

Méthodes sur un objet de type « Set »

- .add(clef)

Permet d'ajouter une clef dans l'objet « Set ».
Si la clef existe déjà, la méthode sera ignorée.

- .has(clef)

Renvoie un booléen pour indiquer si la clef est présente dans l'objet « Set ».

Méthodes sur un objet de type « Set »

- .delete(clef)

Permet de supprimer une clef.
Cette méthode renvoie « true » si l'élément existe et qu'il a bien été supprimé.

- .clear()

Permet de supprimer toutes les clefs.

- .forEach(méthode)

Permet d'effectuer un traitement itératif. *(Identique à la méthode forEach() des Array)*

Propriété sur un objet de type « Set »

La propriété « size » permet d'obtenir le nombre d'éléments de l'objet « Set ».

```
const ensemble = new Set();
const s1 = ensemble.size;    // → 0

ensemble.add('Zaza');
ensemble.add('Riri');
ensemble.add('Della');
const s2 = ensemble.size;    // → 3

ensemble.delete('Riri');
const s3 = ensemble.size;    // → 2

ensemble.clear();
const s4 = ensemble.size;    // → 0
```

Évaluation en court-circuit

Annexes

Évaluation en court-circuit

Le JavaScript permet d'utiliser les opérateurs logiques pour créer un « court-circuit » lors de l'évaluation d'une instruction.

Cela permet de rendre l'exécution d'instruction de code conditionnel.

- Opérateur logique ET : si l'expression est « Truthy », la suite sera interprétée

```
callback && callback();
```

- Opérateur logique OU : si l'expression est « Truthy », la suite ne sera pas interprétée

```
const message = box.lastMessage || 'Aucun message';
```

Les closures

Annexes

C'est quoi une closure ?

Une closure (fermeture) permet à une fonction de conserver l'accès aux variables de son environnement parent, même après que celui-ci ait terminé son exécution.

Dans l'exemple : la fonction retournée garde l'accès à la variable « compteur », bien que « creerCompteur() » ait terminé son exécution.

```
// Exemple d'une closure via un compteur
function creerCompteur() {
  let compteur = 0;

  return function () {
    compteur++;
    return compteur;
  };
}

const incrementer = creerCompteur();

console.log(incrementer()); // 1
console.log(incrementer()); // 2
```

C'est quoi une IIFE ?

Une IIFE *(Immediately Invoked Function Expression)* est une fonction déclarée puis exécutée immédiatement.

Elle permet de créer un scope isolé, pour éviter de polluer le scope global.

```
// Exemple d'une IIFE avec une fonction fléchée
(() => {

  const bouton = document.getElementById('btn-like');

  bouton.addEventListener('click', function () {
    bouton.textContent = "Merci 💖";
    bouton.disabled = true;
  });

})();
```

Le pattern module : IIFE + closure

En combinant IIFE et closure, on peut créer un module :

- une IIFE isole le code d'initialisation et les variables privées
- les fonctions retournées (closures) gardent un accès à ces variables pour former une API publique.

```
// Module qui gère une liste de tâches
const gestionnaireTaches = (() => {
  let taches = [];

  return {
    ajouterTache: (tache) => {
      taches.push(tache);
    },
    compterTaches: () => {
      return taches.length;
    }
  };
})();

gestionnaireTaches.ajouterTache("Réviser le JavaScript");
gestionnaireTaches.ajouterTache("Préparer l'examen");

console.log(gestionnaireTaches.compterTaches());  // 2
```

Le destructuring

Annexes

Objectif du destructuring

Le destructuring est une syntaxe qui permet de réaliser une affectation par décomposition. Celle-ci permet d'extraire des données depuis un objet ou un tableau.

L'objectif du destructuring est donc de pouvoir affecter les valeurs d'une structure de données à des variables en une seule instruction.

```
// Exemple simple

// - Ensemble de données
const fruits = ['Pomme', 'Cerise', 'Mangues'];
const person = { firstname: 'Della', lastname: 'Duck'};

// - Utilisation du destructuring
const [f1, f2, f3] = fruits;
const { firsname, lastname } = person;
```

Décomposition de collection

- Déclaration de variable et initialisation

La valeur des variables est basée sur la position des éléments de la collection.

```
const people = ['Della', 'Balthzazar', 'Zaza', 'Gontran'];

// Affection sans décomposition
const duck01 = people[0];
const duck02 = people[1];
const duck03 = people[2];

// Affection avec décomposition
const [duck01, duck02, duck03] = people;
```

Décomposition de collection

- Affectation sans déclaration

Il est possible d'utiliser le destructuring pour uniquement affecter des variables.

```
const numbers = [42, -5, 13];
let val1, val2, val3;

[val1, val2, val3] = numbers;
```

- Valeurs par défaut

On peut définir une valeur qui sera utilisée si la valeur extraite est « undefined ».

```
const numbers = [-9, 3.14];

const [val1, val2 = 10, val3 = 42] = numbers;
```

Décomposition de collection

- Affecter le reste d'une collection à une variable

Il est possible d'utiliser l'opérateur « rest » avec le destructuring.

```
const people = ['Donald', 'Daisy', 'Riri', 'Fifi', 'Loulou'];

// Affection sans décomposition
const duck01 = people[0];
const duck02 = people[1];
const neveux = people.slice(2);

// Affection avec décomposition
const [duck01, duck02, ...neveux] = people;
```

Décomposition de collection

- Ignorer des valeurs

Il est possible d'ignorer certaines des valeurs de la collection.

```
const numbers = [5, 10, 15, 20, 25, 30, 35, 40, 45, 50];

const [val1, , val3, , , val6, val7] = numbers;
```

- Échange de variables

Le destructuring permet également d'inverser le contenu de variables.

```
let a = 42;
let b = 13;

[a, b] = [b, a];
```

Décomposition d'objet

- Déclaration de variable et initialisation

La valeur des variables est basée sur le nom des clefs de l'objet ciblé.

```
const person = { firstname: 'Della', lastname: 'Duck', job: 'Pilote' };

// Affection sans décomposition
const firstname = person.firstname;
const lastname = person.lastname;
const job = person.job;

// Affection avec décomposition
const { firstname, lastname, job } = person;
```

Décomposition d'objet

- Affectation sans déclaration

Il est possible d'utiliser le destructuring pour uniquement affecter des variables.

```
const course = { title: 'JavaScript', acronym: 'JS', section: 'Dev' };
let title, acronym;

({ title, acronym } = course);
```

- Valeurs par défaut

On peut définir une valeur qui sera utilisée si la valeur de la clef est « undefined ».

```
const course = { title: 'Information', acronym: 'IC' };

const { title, section = 'N/A' } = course;
```

Décomposition d'objet

- Affecter le reste

Il est possible d'utiliser l'opérateur « rest » avec le destructuring.

```
const company = {
  name: 'Bstorm',
  lat: 50.66072,
  lon: 4.632701
};

// Affection sans décomposition
const name = company.name;
const coord = { lat: company.lat, lon: company.lon };

// Affection avec décomposition
const { name, ...coord } = company;
```

Décomposition d'objet

- Affectation avec un nom différent

Il est possible définir un nom de variable différent que celui de la clef ciblé.

```
const person = { firstname: 'Donald', lastname: 'Duck', job: 'Matelot' };

const { firstname: prenom, lastname: nom, job } = person;
```

Il est également possible de définir une valeur par défaut en plus du nom de variable.

```
const position1 = { x: 5, y: -9, z: 2 };
const position2 = { x: 3, y: 6 };

const { x: x1, y: y1, z: z1 = 0 } = position1;
const { x: x2, y: y2, z: z2 = 0 } = position2;
```

Décomposition d'objet

- Décomposition imbriquée

```
const user = {
  id: 42,
  pseudo: 'zavand',
  name: {
    firstname: 'Zaza',
    lastname: 'Vanderquack',
  },
  langagues: [
    { code: 'en', name: 'English', isFav: true },
    { code: 'fr', name: 'French' },
  ]
};

const { name : { firstname }, langagues: [{ name: lang1}, { name: lang2}] } = user;
```

Décomposition d'objet

- Décomposition d'un paramètre d'une fonction

Lorsqu'on créer une méthode, il est possible d'utiliser le destructuring pour définir le nom des paramètre de l'objet attendu et si nécessaire la valeur par défaut de ceux-ci.

```
function showBook({ title, author, category = 'Others' }) {
  // ...
}

const book1 = {title: 'Tout JavaScript', author: 'Olivier Hondermarck', category: 'Dev' };
showBook(book1);

const book2 = { title: 'Le Petit Prince', author: 'Antoine de Saint-Exupéry' };
showBook(book2);
```

Web Storage

Annexes

C'est quoi le Web Storage ?

Le Web Storage est une API du navigateur qui permet de stocker des données de type clef/valeur directement côté client, sans passer par un serveur.

Elle se compose de deux objets :

- **localStorage** : les données persistent même après la fermeture du navigateur.
- **sessionStorage** : les données ne persistent que le temps de l'onglet ouvert (supprimées à la fermeture).

Les deux objets partagent la même API (mêmes méthodes et propriétés), seule la durée de vie des données diffère.

Remarque : les données sont stockées par origine (protocole + domaine + port).

Le localStorage VS le sessionStorage

localStorage

- Persiste après la fermeture du navigateur.
- Partagé entre tous les onglets/fenêtres de la même origine.
- Pas de date d'expiration automatique (suppression manuelle nécessaire).

sessionStorage

- Supprimé à la fermeture de l'onglet.
- Propre à chaque onglet, même si l'origine est identique.
- Utile pour des données temporaires (ex : état d'un formulaire en cours).

Comparatif avec le cookie

Critère	Cookie	Web Storage
Capacité de stockage	~4 Ko	~ 5-10 Mo (selon navigateur)
Envoi au serveur	Envoyé à chaque requête HTTP	Jamais envoyé automatiquement
Expiration	Configurable (date d'expiration)	localStorage : jamais / sessionStorage : fermeture onglet
Accessibilité	JS (sauf « HttpOnly ») et serveur	JS uniquement
Usage typique	Authentification, tracking serveur	Préférences utilisateur, cache local, état d'UI

Remarque : contrairement au cookie, le Web Storage ne surcharge pas les requêtes HTTP puisqu'il n'est jamais transmis automatiquement au serveur.

Les limites du Web Storage

- Capacité limitée (généralement 5 à 10 Mo par origine, selon le navigateur).
- Stockage uniquement synchrone : peut bloquer le thread principal sur de gros volumes de données.
- Les données sont stockées en clair (aucun chiffrement) : ne jamais y stocker d'informations sensibles (mots de passe, tokens sensibles, données personnelles).
- Accessible à tout script JavaScript de la page (risque en cas de faille XSS).
- Ne fonctionne qu'avec des chaînes de caractères : toute autre donnée doit être convertie (ex : JSON.stringify).
- Pas de mécanisme d'expiration natif pour localStorage.

Les méthodes de l'API Web Storage

Les objets **localStorage** et **sessionStorage** partagent les mêmes méthodes :

- .setItem(clef, valeur) : Ajoute ou met à jour une valeur associée à une clef.
- .getItem(clef) : Renvoie la valeur associée à la clef, ou null si elle n'existe pas.
- .removeItem(clef) : Supprime l'élément associé à la clef.
- .clear() : Supprime tous les éléments du storage.
- .key(index) : Renvoie le nom de la clef à l'index donné.
- .length : Propriété qui renvoie le nombre d'éléments stockés.

Exemple d'utilisation simple

```
// Ajouter une valeur
localStorage.setItem('pseudo', 'Della');

// Récupérer une valeur
const pseudo = localStorage.getItem('pseudo');
console.log(pseudo); // "Della"

// Supprimer une valeur
localStorage.removeItem('pseudo');

// Vérifier si une clef existe
const pseudoRecup = localStorage.getItem('pseudo')
console.log(pseudoRecup); // null
```

Exemple d'utilisation avec un objet

```
// Objet JS à stocker
const utilisateur = {
  nom: "Duck",
  prenom: "Della",
  age: 25
};

// Stocker l'objet (conversion en chaîne JSON)
localStorage.setItem('utilisateur', JSON.stringify(utilisateur));

// Récupérer et reconvertir l'objet
const dataJson = localStorage.getItem('utilisateur')
const utilisateurRecupere = JSON.parse(dataJson);

console.log(utilisateurRecupere.prenom); // "Della"
```

L'événement storage

L'événement storage se déclenche sur l'objet window lorsqu'une donnée est modifiée dans le localStorage, depuis un autre onglet ou une autre fenêtre de la même origine.

Particularités :

- Il ne se déclenche pas dans l'onglet qui a lui-même modifié la donnée.
- Il ne concerne pas sessionStorage (propre à chaque onglet).
- Il permet de synchroniser plusieurs onglets ouverts sur la même origine.

```
window.addEventListener('storage', (event) => {
  console.log(event.key);       // Clef modifiée
  console.log(event.oldValue);  // Ancienne valeur
  console.log(event.newValue);  // Nouvelle valeur
  console.log(event.url);       // URL du document à l'origine du changement
});
```

Bonnes pratiques

- Ne jamais stocker de données sensibles (mots de passe, données bancaires...).
- Toujours vérifier la disponibilité du storage avant utilisation (certains navigateurs en navigation privée peuvent le restreindre).
- Encapsuler les accès (getItem/setItem) dans un try...catch, le quota pouvant être dépassé (QuotaExceededError).
- Préférer sessionStorage pour des données temporaires liées à une session de navigation.
- Passer par JSON.stringify / JSON.parse pour stocker des objets ou des tableaux.
- Nettoyer régulièrement les données obsolètes pour éviter d'atteindre le quota.
- Ne pas utiliser le Web Storage comme unique source de vérité pour des données critiques : privilégier une synchronisation avec le serveur si nécessaire.

L'API de validation des formulaires

Annexes

Objectif de l'API de validation des contraintes

Les données d'un formulaire peuvent être validé par règle définie dans la balise html telle que : required, minlength, maxlength, pattern, etc...

Si vous souhaitez ajouter des règles custom, il est nécessaire d'utiliser le JavaScript.

L'API de validation des contraintes permet d'ajouter facilement des règles de validation sans devoir gérer manuellement le comportement du navigateur.

Elle permet :

- D'obtenir l'état de validation des champs.
- De définir un champs en erreur.

Les propriétés de l'API de validation des contraintes

- .validity

Un objet avec l'états (booléen) des différentes contraintes à valider

- .validationMessage

Le message d'erreur lorsqu'une contrainte est invalide.
Sinon, la propriété contient une chaîne de caractère vide

- .willValidate

Valeur booléen qui indique si l'élément doit être validé.

Les méthodes de l'API de validation des contraintes

- .checkValidity()

La méthode renvoi « true » si l'élément n'a pas de problème de validation.
Sinon, elle renvoi « false » et déclenche également l'évent « invalid ».

- .setCustomValidity(message)

Permet de définir une erreur personnalisé sur un élément avec son message d'erreur.
Cet élément sera considéré comme invalide par le formulaire.

Si l'argument est une chaîne de caractères vide, elle supprime l'erreur personnalisé.

Exemple d'erreur personnalisé sur un champ

```
form1['email'].addEventListener('input', (event) => {
  const field = event.currentTarget;
  const email = field.value.toLowerCase();
  const validityState = field.validity;

  if (validityState.typeMismatch) {
    field.setCustomValidity('Votre email est invalide !');
  }
  else if (email.includes('gripsou')) {
    field.setCustomValidity('Vous n\'êtes pas autorisé !');
  }
  else {
    field.setCustomValidity('');
  }
});
```

Références

Références

- MDN Web Docs

https://developer.mozilla.org/fr/docs/Web/JavaScript

- Ecma International's TC39

https://tc39.es/

https://github.com/tc39

Merci pour votre attention.

